Depth, Oversmoothing, and Architecture: A Comparative Study of Graph Neural Networks on the Cora Citation Network

CSE 705 Deep Learning

McMaster University

Kiarash Ara
arak@mcmaster.ca

Yiheng Wu
wu1261@mcmaster.ca

2026-08-03

Abstract

Graph neural networks (GNNs) built from iterative message passing lose the ability to discriminate between nodes as depth increases, a phenomenon known as oversmoothing; this limits the receptive field available to applications such as recommendation on user-item graphs, community detection in social networks, and molecular property prediction, where relevant structure can lie several hops away. We compare four message-passing architectures: GCN, GraphSAGE, GAT, and GCNII, on the Cora citation network across depths 2, 4, 8, 16, and 32, using Dirichlet energy and mean average distance (MAD) as two complementary oversmoothing diagnostics. We capture both quantities at initialization, at the best-validation checkpoint, and at the final training epoch, which lets us separate a collapse attributable to the propagation operator from one attributable to training. Test accuracy degrades sharply beyond depth 4 for GCN, GraphSAGE, and GAT, but through distinct mechanisms rather than one shared failure: GraphSAGE and GAT do not train at all at depth 32, while GCN partially trains and develops a signature of early-layer energy collapse paired with late-layer growth. GCNII, whose identity mapping and initial-residual connection are designed against this dynamic, is the only architecture whose accuracy improves with depth. Among four tested mitigations, Jumping Knowledge restores the most deep-network performance (73.1% test accuracy at depth 32, against a 24.1% unmitigated baseline), while residual connections alone do not clearly outperform the unmitigated baseline at this depth.

Table of Contents

1 Introduction

1.1 Motivation

Many real-world systems are better represented as graphs than as collections of independent observations. In a citation network, for example, papers are represented as nodes and citations are represented as edges. Recommendation systems can connect users to products through ratings or purchases, while molecular graphs represent atoms and the chemical bonds between them. In each case, the relationships between objects contain useful information that would be lost if every object were considered separately.

Graph representation learning uses both node information and graph structure to learn useful numerical representations. These representations, usually called node embeddings, can then be used for tasks such as node classification, link prediction, community detection, and recommendation. Graph neural networks (GNNs) are especially useful for this purpose because they allow connected nodes to exchange information through a process known as message passing.

During message passing, each node updates its representation by combining its own features with information received from its neighbors. A single layer uses information from nearby nodes, while stacking several layers allows information to travel across a larger part of the graph. In principle, this should help a model capture relationships that are more than one step away. In practice, however, adding more layers often makes GNNs harder to train and can reduce their ability to distinguish between nodes.

Depth is useful because each additional layer increases the receptive field of a node: one message-passing layer reaches direct neighbors, two reach neighbors-of-neighbors, and an L-layer model can draw on information from roughly L hops away. This can be important in applications where relevant information is not located in the immediate neighborhood. At the same time, repeated aggregation can make node representations increasingly similar. When this happens, nodes from different classes may become difficult to separate. This behavior is commonly referred to as oversmoothing (Li et al. 2018; Oono and Suzuki 2020).

Depth is not optional in every domain that motivates this study. In physics-informed graph neural networks, which represent a discretized physical domain (a mesh, for example) as a graph and propagate a physical state across it, a useful prediction requires information to reach across enough of the domain, so oversmoothing is a first-order obstacle rather than a tunable inconvenience. Cora is small and tightly clustered compared with such a domain, and the specific numbers reported in this study are not expected to transfer to a physics mesh, which is far larger and requires information to travel much farther. What is expected to transfer is the diagnostic methodology, the measurement apparatus built around Dirichlet energy and MAD, and the qualitative behavior of the mitigations, not the numbers themselves.

1.2 The Task: Semi-Supervised Node Classification

This project studies semi-supervised node classification on the Cora citation network. Cora contains 2,708 scientific papers connected by citation links. Each paper is represented by a 1,433-dimensional binary bag-of-words feature vector and belongs to one of seven research areas (Sen et al. 2008; Yang et al. 2016). The standard public split contains 140 labeled training nodes, 500 validation nodes, and 1,000 test nodes. The features and graph positions of all nodes are available during training, but only the labels of the training nodes are used to update the model.

A two-layer Graph Convolutional Network (GCN) is known to perform well on this dataset (Kipf and Welling 2017). Reproducing that baseline is an important first step, but it does not explain what happens when the network becomes deeper. For this reason, depth is the main experimental variable in this study. We compare models with 2, 4, 8, 16, and 32 message-passing layers and examine how their classification performance and hidden representations change as more layers are added.

1.3 Thesis and Methodology

Oversmoothing is often discussed as though it were the only reason deep GNNs fail, but a decrease in accuracy does not prove that oversmoothing has occurred. A deep model may also fail because optimization becomes ineffective, gradients become unstable, or too much neighborhood information is compressed into a fixed-size representation. These mechanisms can produce similar changes in classification accuracy while affecting the hidden representations in different ways.

The purpose of this study is therefore not simply to show that deeper GNNs perform worse. Instead, we investigate how different architectures fail as their depth increases and whether the failures share the same cause. Three standard message-passing architectures are compared: GCN (Kipf and Welling 2017), GraphSAGE (Hamilton et al. 2017), and the Graph Attention Network (GAT) (Veličković et al. 2018). GCN uses degree-normalized aggregation, GraphSAGE combines a node's own representation with the mean representation of its neighbors, and GAT learns attention weights for individual connections. Because their aggregation rules differ, they may also respond differently to increasing depth.

The models are evaluated using test accuracy, micro-F1, and macro-F1. Micro-F1 is included because it is required by the project specification, although it is equal to accuracy in a single-label multiclass problem. Macro-F1 is also reported because it gives equal importance to each class and can reveal whether a model is performing poorly on less accurately predicted classes.

Classification metrics alone do not describe what is happening inside a deep network. We therefore examine the hidden node representations using two oversmoothing measures: Dirichlet energy and mean average distance (MAD). Dirichlet energy measures how much connected nodes differ from each other over the graph, while MAD measures the average cosine distance between node representations. The two metrics are used together because they capture different aspects of representation collapse.

The metrics are recorded at three stages of every run. The first measurement is taken at initialization, before any gradient update. The second is taken at the checkpoint with the lowest validation loss, which is also the checkpoint used for the reported test results. The third is taken at the final training epoch before the best checkpoint is restored. Separating these three stages helps us distinguish behavior caused by the untrained propagation process from changes that develop during training.

We also compare several methods designed to improve deep GNNs. Residual connections preserve information from earlier layers. PairNorm attempts to prevent the node representations from collapsing toward one another (Zhao and Akoglu 2020). Jumping Knowledge combines representations learned at different depths (Xu et al. 2018). GCNII uses initial residual connections and identity mapping to support much deeper graph networks (M. Chen et al. 2020). These methods are compared using both classification results and representation-level measurements.

The experiments use the same Cora split and a common training protocol across the main model comparisons. Each main configuration is repeated using ten fixed random seeds. The hidden width, activation function, dropout rate, optimizer, and model-selection rule are kept consistent wherever

possible. This reduces the chance that an apparent depth effect is actually caused by a different training setup. A separate two-layer GCN experiment is also used to check that the implementation reproduces the established Cora baseline.

1.4 Findings and Scope

The proposal that initiated this project anticipated oversmoothing as a mechanism shared across architectures, with node representations expected to collapse progressively as depth increased. The results in this report support a narrower, more specific claim instead, and that narrowing is itself part of the study's contribution: representational collapse toward a low-energy state is visible cleanly at initialization, before any training occurs, but it does not survive training in the same form under this study's hyperparameters.

The central argument of this report is that depth-related failure is architecture-dependent. GCN, GraphSAGE, and GAT all lose classification performance as they become deeper, but the internal behavior of the models is not the same. The results show that deep GraphSAGE and GAT remain close to strongly collapsed initialization states and show little evidence of effective training. Deep GCN behaves differently: it trains partially and develops strong contraction in its early layers together with energy growth in later layers. GCNII does not follow either pattern and is the only tested architecture whose classification performance improves at the largest depths.

This study is limited to the Cora dataset and its standard public split. Therefore, the exact depth at which performance declines and the ranking of the mitigation methods should not be assumed to transfer directly to larger, noisier, or less homophilous graphs. The project also does not record per-layer gradient or weight norms, so some explanations of the training behavior remain supported by indirect evidence rather than measured directly. The conclusions are therefore limited to what can be established from the classification results, training curves, and representation metrics collected in the experiments.

1.5 Contributions

The main contributions of the project are:

1. A controlled comparison of GCN, GraphSAGE, and GAT across depths from 2 to 32 layers.
2. A layer-wise analysis of node representations using Dirichlet energy and MAD.
3. A comparison of representations at initialization, the selected checkpoint, and the final training epoch.
4. An empirical distinction between different depth-related failure patterns rather than treating every failure as the same form of oversmoothing.
5. An evaluation of residual connections, PairNorm, Jumping Knowledge, and GCNII as methods for improving deep GNN performance.
6. A reproducible experimental pipeline using fixed random seeds, automated result records, validation-based checkpoint selection, and generated tables and figures.

1.6 Report Roadmap

The remainder of the report is organized as follows. Section 2 reviews the background literature on graph representation learning, deep GNNs, oversmoothing, trainability, and mitigation methods.

Section 3 introduces the mathematical notation, model formulations, diagnostic metrics, classification measures, and the Cora dataset. Section 4 describes the implementation and experimental design. Section 5 explains how the main design decisions are represented in the source code. Section 6 presents and discusses the experimental results. Section 7 outlines the limitations of the study and directions for future work, followed by the final conclusions.

2 Problem Statement, Review, and Background

2.1 Problem Statement

The task considered in this project is semi-supervised node classification on the Cora citation network. Each node represents a scientific paper, each edge represents a citation relationship, and the goal is to assign every paper to one of seven research categories. The difficulty is that only 140 of the 2,708 nodes are labeled for training. A successful model must therefore learn from both the words associated with each paper and the citation structure connecting the papers (Sen et al. 2008; Yang et al. 2016).

Graph neural networks are well suited to this problem because they allow information to move between connected nodes. Instead of classifying each paper from its own feature vector alone, a GNN can also use information from papers that cite it or are cited by it. This is useful on a citation network because connected papers often discuss related topics.

A shallow GNN can perform well on Cora, but this does not fully answer the question studied in this report. The main problem is to understand what happens when more message-passing layers are added. A deeper network can use information from more distant parts of the graph, but it may also become difficult to train or lose the differences between node representations. The practical question is therefore not simply whether depth helps or hurts. It is whether different GNN architectures respond to depth in the same way, and whether poor deep-model performance is actually caused by oversmoothing.

This distinction matters because several different problems can produce a similar decrease in test accuracy. A network may train successfully and then produce overly similar node embeddings. It may receive gradients that are too weak to update its earlier layers. It may compress too much distant information into a fixed-size representation, or it may stop near its randomly initialized state without learning a useful decision boundary. Looking only at the final accuracy would treat all of these situations as the same failure even though their causes and possible solutions are different.

The project addresses this problem by comparing GCN, GraphSAGE, and GAT under the same depth grid and training protocol. GCNII is also included as an architecture designed specifically for deeper graph networks. Classification metrics are examined together with layer-wise representation metrics so that a decrease in accuracy is not automatically labeled as oversmoothing.

2.2 Message Passing and the Role of Depth

Most modern GNNs can be understood through the message-passing framework (Gilmer et al. 2017). At each layer, a node collects information from its neighbors, combines that information with its current representation, and passes the result to the next layer. Although the exact aggregation rule differs between architectures, the general idea is the same.

Depth controls how far information can travel. After one message-passing layer, a node has access to information from its direct neighbors. After two layers, it can also receive information that

originated from neighbors of those neighbors. Continuing this process expands the receptive field of the node.

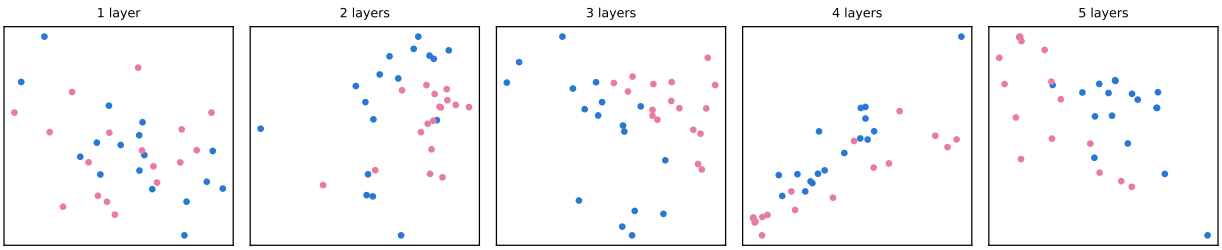
This larger receptive field is the main reason to consider deep GNNs. In many graphs, the information needed for a prediction may not be available within one or two hops. A recommendation may depend on several stages of interaction between users and items. A social-network prediction may depend on membership in a wider community rather than on immediate friendships alone. In a molecular or physical graph, distant components may also affect the property being predicted.

However, a larger receptive field is only useful when the network can preserve and organize the information it receives. Each message-passing layer mixes the representations of connected nodes. Repeating this operation many times can remove useful differences between nodes, especially when the aggregation rule behaves like an averaging process. The model then gains access to a wider neighborhood but may lose the ability to tell the nodes in that neighborhood apart.

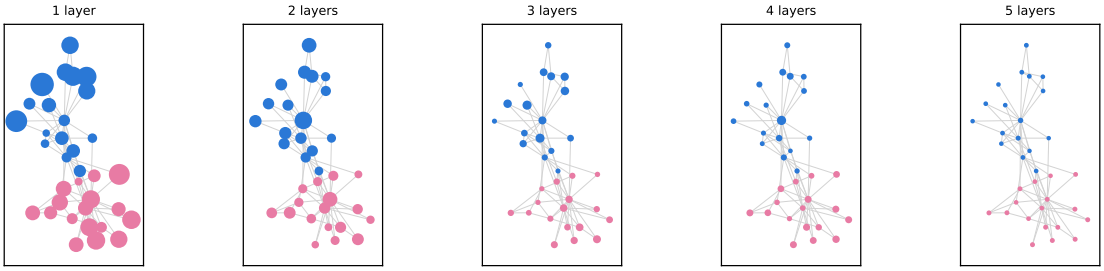
This produces the main tension studied in the project. Increasing depth can provide more structural information, but it can also weaken the class-discriminative information already present in the node embeddings.

2.3 Oversmoothing in Graph Neural Networks

Oversmoothing describes the tendency of node representations to become less distinguishable as message passing is repeated. Li, Han, and Wu connected graph convolution with Laplacian smoothing and showed that the same operation that helps nearby nodes share information can eventually make their representations too similar (Li et al. 2018). Oono and Suzuki later showed that, under suitable conditions, deep GNN representations approach a restricted subspace as depth increases, reducing the expressive power available for node classification (Oono and Suzuki 2020).



(a) Raw two-dimensional embedding-space output, no post-hoc projection, color marking the two-faction split.



(b) The same computation viewed at each node's fixed graph position (one spring layout, held constant across panels); marker size is each node's Euclidean deviation from that depth's embedding centroid, on one shared scale across all five panels so a shrinking marker is itself a collapse signal.

Figure 1: Embedding collapse in an untrained, randomly initialized GCN stack at depths 1 through 5, applied to Zachary's karate club network with one-hot node identity features. No backpropagation is performed; the illustration isolates the effect of repeated graph convolution alone, the Laplacian-smoothing mechanism (Li et al. 2018) describes.

At depth 1, the two factions are not visibly separated (Figure 1a); a single aggregation step mixes each node's identity with only its immediate neighbors', which random weights do not yet turn into a faction-aligned direction. By depth 4, the embeddings compress toward a single, nearly linear direction, a loss of rank rather than a collapse to one point, and depth 5 keeps that same low-dimensional arrangement.

This collapse is uneven across nodes, not a uniform shrinkage (Figure 1b): most nodes are already near-invisible by depth 2-3, while a handful, visibly the higher-degree ones, retain a noticeably larger marker through depth 5. Node degree correlates with retained depth-5 deviation at Pearson $r = 0.37$ (34 nodes, one seed, one graph). This is consistent with, not independent confirmation of, the $\sqrt{\text{degree}}$-weighted null-space direction the threats-to-validity discussion in Section 6 describes for GCN's own collapse on Cora.

The intuitive explanation is straightforward. Suppose two connected nodes have different feature vectors. After one aggregation step, each representation contains part of the other's information. After many similar steps, the difference between them can become much smaller. Across a complete graph, this process can cause many node embeddings to point in similar directions or to vary only within a low-dimensional space.

This becomes a classification problem when nodes from different classes are also smoothed together. The classifier receives less information with which to separate the classes, even if the graph structure was initially helpful. Oversmoothing therefore creates a depth limit: beyond a certain point, adding

layers may reduce rather than improve the usefulness of the learned representations.

It is important, however, not to define oversmoothing only as a decrease in accuracy. Accuracy is an output-level measurement. It shows whether the final predictions are correct but does not show what happened inside the network. A model may have low accuracy without its hidden representations becoming similar, and a representation may become smoother without immediately causing a large accuracy decrease.

For this reason, oversmoothing should be examined directly from the hidden embeddings. Previous work has proposed measures based on pairwise distances, graph smoothness, and the geometry of the representation space. The MAD measure, for example, uses cosine distance to describe how far node representations are from one another (D. Chen et al. 2020). Dirichlet energy instead measures variation over the graph and becomes smaller when connected nodes move toward a smooth representation. The mathematical definitions of both measures are given in Section 3.

The two metrics do not measure exactly the same property. MAD is mainly concerned with the directions of node embeddings, while Dirichlet energy also depends on how the representations vary relative to the graph. A set of embeddings can become nearly one-dimensional without reaching the null space of the graph Laplacian. In that case, MAD may indicate strong collapse while Dirichlet energy remains nonzero. Using both measures provides a more careful view than relying on either one alone.

2.4 Oversmoothing, Trainability, and Oversquashing

Oversmoothing is only one possible difficulty in a deep GNN. Another is trainability. Adding layers creates a longer path between the loss and the earlier model parameters. If useful gradient information does not reach those parameters, the early layers may remain close to their initial values. The model may then produce poor predictions because it did not train effectively, not because training gradually pushed its embeddings into an oversmoothed state.

Kipf and Welling report a depth study in an appendix rather than in their main results: testing depths from one to ten layers, with and without residual connections, they find the best performance at two to three layers and training becoming difficult past roughly seven layers without residual connections on citation networks (Kipf and Welling 2017). This is a statement about optimization difficulty, not about representation collapse, and it is the specific prior-art finding this study's central architecture-dependent result builds on. The appendix experiment uses five-fold cross-validation over all labeled nodes rather than the 140-label standard split used throughout this study, so its numbers are qualitative precedent for a trainability limit at depth, not a directly comparable baseline.

This difference can be difficult to identify from one final checkpoint. A network that barely moved from initialization and a network that trained before collapsing may both have low accuracy and low embedding diversity. Their solutions would be different. The first may require changes to optimization, initialization, or gradient flow. The second may require a mechanism that preserves representation differences during propagation.

Oversquashing is another related but separate problem. As depth increases, the number of nodes that can influence a representation may grow rapidly. All of this information must still be compressed into a hidden vector of fixed width. Important signals from distant nodes may therefore be weakened or lost before they reach the target node (Topping et al. 2022). Oversquashing concerns the compression and transmission of distant information, while oversmoothing concerns the loss of distinction among node representations.

These mechanisms can occur together. A deep model may have similar node embeddings, weak gradient flow, and difficulty transmitting distant information at the same time. The terms should nevertheless not be treated as interchangeable. Doing so would make it impossible to determine which intervention is appropriate.

The experimental design in this project helps separate these possibilities by capturing the representations at three stages. The initialization capture shows the behavior of the untrained network. The selected-checkpoint capture shows the state used to produce the reported test results. The final-epoch capture shows where training ended before the best checkpoint was restored. Comparing these stages gives evidence about whether a failure was already present at initialization or developed during optimization.

2.5 Architectural Differences

The project compares architectures that use different forms of message passing. These differences provide a useful test of whether depth-related failure is a general property of GNNs or whether it depends on the aggregation rule.

GCN uses a symmetrically normalized adjacency operator to combine the representations of neighboring nodes (Kipf and Welling 2017). The normalization accounts for node degree and makes the operation efficient on a sparse graph. At the same time, repeated normalized aggregation has a clear smoothing effect, which makes GCN an important model for studying oversmoothing.

GraphSAGE was introduced as an inductive framework that learns an aggregation function rather than a separate embedding for every node (Hamilton et al. 2017). Its mean-aggregation form treats the central node and the average of its neighbors through separate transformations. GraphSAGE can also use sampled neighborhoods, which makes it more practical than full-batch GCN on large graphs. In this project, Cora is small enough for full-graph training, so GraphSAGE is mainly used to study how its aggregation rule behaves with depth.

GAT replaces fixed degree-based aggregation weights with learned attention coefficients (Veličković et al. 2018). In principle, this allows the model to give more weight to useful neighbors and less weight to irrelevant ones. However, attention does not guarantee that deep representations remain distinct. A GAT still combines information repeatedly, and the attention mechanism introduces additional parameters and optimization complexity.

These architectures are not expected to have identical depth behavior. Their normalization rules, parameterizations, and initializations differ. A fair comparison must therefore hold the surrounding experimental conditions as constant as possible while recognizing that a shared training setup may not be the individually optimal setup for every architecture.

2.6 Approaches for Improving Deep GNNs

Several methods have been proposed to make deep GNNs easier to train or less prone to representation collapse. They work through different mechanisms, so their effects should not be judged from accuracy alone.

Residual connections allow a layer to retain information from an earlier representation rather than replacing it completely. This creates a direct path through the network and can improve gradient flow. It may also reduce the loss of node-specific information during repeated aggregation. A

residual connection does not directly control the distance between node embeddings, however, so it does not guarantee that oversmoothing will be prevented.

PairNorm takes a more direct approach by normalizing the node embeddings so that their overall dispersion does not continuously shrink (Zhao and Akoglu 2020). Instead of changing the graph convolution itself, it changes the geometry of the representation after each layer. This makes it particularly relevant when the main problem is that embeddings are moving too close together.

Jumping Knowledge avoids relying only on the deepest representation (Xu et al. 2018). It combines information from several layers, allowing the final prediction to use both shallow and deep features. Shallow layers retain more local information, while deeper layers contain information from larger neighborhoods. A model can therefore use the depth that is most useful without discarding every earlier representation.

GCNII changes the architecture more fundamentally. It repeatedly reintroduces the initial node representation and uses an identity-mapping term to limit how strongly each layer transforms its input (M. Chen et al. 2020). These choices are designed to preserve the original features and support much deeper graph networks. Unlike residual connections, PairNorm, and Jumping Knowledge in this project, GCNII is treated as a separate architecture rather than as a hook attached to a standard GCN.

The methods also introduce different costs. Jumping Knowledge requires a readout that combines several layers. GCNII uses additional projections and architecture-specific parameters. PairNorm adds normalization but no learned weights. Residual connections are simple when the layer widths match but may require projections otherwise. These differences mean that classification accuracy, representation behavior, model capacity, and computational cost all need to be considered when the methods are compared.

2.7 Large-Scale and Noisy Graphs

Cora is useful for controlled experiments because it is small enough to train in full batch and to measure every node representation at every layer. Many real graphs are much larger. A recommendation network, transaction graph, or social platform may contain millions of nodes and edges. In these settings, storing the full graph and all intermediate embeddings in memory may not be possible.

Neighbor sampling is one common response to this problem. GraphSAGE, for example, can train on sampled neighborhoods instead of expanding every neighbor at every layer (Hamilton et al. 2017). This reduces memory and computation, but it also changes the message-passing process. A node may receive a different sampled neighborhood from one training step to another. Metrics such as Dirichlet energy and MAD must then be interpreted carefully because the graph used for training may not be the same graph used for measurement.

Large graphs also make the depth problem more important. In a small, well-connected citation network, several hops may already cover a substantial part of the graph. In a large sparse graph, a useful signal may genuinely be many hops away. A shallow architecture may not reach it, while a deep architecture may oversmooth, oversquash, or become too difficult to optimize before the signal arrives.

Noise creates another challenge. Real citation, social, recommendation, and transaction networks may contain missing, incorrect, or irrelevant edges. Message passing does not automatically know

whether an edge is reliable. Once incorrect information is aggregated, additional layers can spread it farther through the graph. Noisy node features can create a similar effect because their influence is shared with neighboring nodes.

GAT may appear naturally suited to noisy graphs because it can assign different weights to neighbors, but learned attention is not a guarantee that unreliable edges will be ignored. Attention coefficients are learned from the same limited data as the rest of the model and may themselves be unstable. Normalization, robust aggregation, graph denoising, and uncertainty-aware edge weighting are possible directions, but they are outside the empirical scope of the present Cora study.

The exact numerical findings from Cora should therefore not be assumed to transfer directly to large or noisy graphs. What may transfer more reliably is the evaluation method: compare initialized and trained representations, use more than one smoothing metric, keep the representation band comparable across architectures, and distinguish failure to optimize from collapse that develops during training.

2.8 Gap Addressed by This Study

The same accuracy decline with depth can represent different internal failures, and a method that helps one failure may not help another; accuracy alone does not distinguish them.

This study addresses that gap through a controlled comparison with three main features.

First, architecture and depth are examined together: GCN, GraphSAGE, and GAT are evaluated across the same depths. GCNII provides an additional comparison with an architecture designed specifically for depth.

Second, the study combines output-level and representation-level evidence. Accuracy and macro-F1 describe predictive performance; Dirichlet energy and MAD describe changes in the hidden embeddings.

Third, the representation metrics are captured before training, at the selected checkpoint, and at the final epoch, which makes it possible to ask whether a deep model learned and then deteriorated or whether it remained close to a poor initial state.

The scope remains deliberately limited. The project studies one dataset, one public split, and a coordinated training protocol. It does not claim to identify a universal depth limit for GNNs or to establish that one mitigation is always best. Its goal is narrower: to show how a careful combination of architecture comparison, training evidence, and representation diagnostics can give a more accurate account of why deep GNNs fail.

3 Theory and Datasets

This section introduces the mathematical ideas used in the project and explains how they relate to the Cora node-classification task. The goal is to define the graph, the learning problem, the four GNN architectures, and the measurements used to study what happens as the networks become deeper.

3.1 Graph Notation

A graph is written as

$$G = (V, E),$$

where V is the set of nodes and E is the set of edges. In the Cora citation network, each node is a scientific paper and each edge represents a citation relationship.

Let

$$N = |V|$$

be the number of nodes. The graph structure can be stored in an adjacency matrix

$$A \in \{0, 1\}^{N \times N},$$

where

$$A_{ij} = \begin{cases} 1, & \text{if nodes } i \text{ and } j \text{ are connected,} \\ 0, & \text{otherwise.} \end{cases}$$

Although citations are directed in the original data, the processed Cora graph used in this project is treated as undirected for message passing. Therefore, if node i is connected to node j, information can travel in both directions.

Each node also has an input feature vector. The full node-feature matrix is

$$X \in \mathbb{R}^{N \times d},$$

where d is the number of features and the ith row, x_i, contains the features of node i. For Cora, these features form a binary bag-of-words representation of each paper.

The class label of node i is written as

$$y_i \in \{1, \dots, C\},$$

where C is the number of classes. Cora contains seven research-topic classes.

The nodes are divided into three disjoint sets:

$$V_{\text{train}}, \qquad V_{\text{val}}, \qquad V_{\text{test}}.$$

The training labels are used to update the model. The validation labels are used for model selection and early stopping. The test labels are kept separate and are used only to report the final performance.

3.2 Self-Loops and Graph Normalization

A GNN should usually preserve a node's own information when it gathers information from its neighbors. For this reason, self-loops are added to the adjacency matrix:

$$\widetilde{A} = A + I,$$

where I is the identity matrix.

The degree of node i in the augmented graph is

$$\tilde{d}_i = \sum_{j=1}^{N} \widetilde{A}_{ij}.$$

The corresponding degree matrix is

$$\widetilde{D} = \operatorname{diag}\left(\tilde{d}_1, \ldots, \tilde{d}_N\right).$$

A common normalized propagation matrix is

$$\widehat{A} = \widetilde{D}^{-1/2}\widetilde{A}\widetilde{D}^{-1/2}.$$

This normalization reduces the effect of differences in node degree. Without it, nodes with many neighbors could produce messages with much larger magnitudes than nodes with only a few neighbors.

The normalized graph Laplacian is

$$\mathcal{L} = I - \widehat{A}.$$

The Laplacian measures how much a signal changes across the graph. A signal that has similar values on connected nodes has low Laplacian energy, while a signal that changes strongly across edges has high energy.

3.3 The Null Space of the Normalized Laplacian

The null space of the normalized Laplacian is important for understanding oversmoothing. For a connected graph,

$$\mathcal{L}\widetilde{D}^{1/2}\mathbf{1} = 0,$$

where $\mathbf{1}$ is the all-ones vector. This means that the zero-energy direction is proportional to

$$\widetilde{D}^{1/2}\mathbf{1}.$$

The entries of this vector are the square roots of the augmented node degrees. Therefore, on an irregular graph, a completely smooth representation under the normalized Laplacian is not necessarily constant across nodes. Instead, it is weighted by the square root of node degree.

This distinction matters in this project because Cora is not a regular graph. Its nodes do not all have the same degree. A representation may become one-dimensional and point in the constant direction while still having nonzero normalized Dirichlet energy. This is one reason why Dirichlet energy and MAD are both needed: the two metrics can respond differently to the same representation geometry.

A direct consequence follows for how "collapse" should be read under this metric. A representation that fully collapses under Dirichlet energy, in the sense of converging onto this null space rather than merely becoming similar across nodes, is converging toward a representation proportional to the square root of node degree, not toward a single constant vector. High-degree nodes therefore retain a larger-magnitude representation than low-degree nodes even at total collapse under this metric. "Collapsed," in this report, does not mean that every node's representation becomes numerically identical.

3.4 Semi-Supervised Node Classification

A GNN with parameters θ receives the feature matrix and graph structure and produces one output vector for each node:

$$Z = f_\theta(X, A),$$

where

$$Z \in \mathbb{R}^{N \times C}.$$

The row Z_i contains the class scores, or logits, for node i. The logits are converted into class probabilities using the softmax function:

$$\widehat{Y}_{ic} = \frac{\exp(Z_{ic})}{\sum_{k=1}^{C} \exp(Z_{ik})}.$$

The predicted class is

$$\hat{y}_i = \underset{c \in \{1,\ldots,C\}}{\arg\max}\ \widehat{Y}_{ic}.$$

Although predictions are produced for every node, the training loss is calculated only on the labeled training nodes. Using one-hot encoded labels Y_{ic}, the cross-entropy loss is

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|V_{\text{train}}|} \sum_{i \in V_{\text{train}}} \sum_{c=1}^{C} Y_{ic} \log\left(\widehat{Y}_{ic}\right).$$

This setting is called semi-supervised because only a small portion of the nodes have labels available for training.

It is also transductive. The features and graph positions of the validation and test nodes are available while the model is trained, but their labels do not contribute to the loss.

3.5 Message Passing

Most GNNs can be described using a message-passing framework (Gilmer et al. 2017). Let $h_i^{(\ell)}$ be the representation of node i at layer ℓ. The input representation is

$$h_i^{(0)} = x_i.$$

A general message-passing layer first collects information from the neighbors of node i:

$$m_i^{(\ell)} = \text{AGGREGATE}^{(\ell)}\left(\left\{h_j^{(\ell)} : j \in \mathcal{N}(i)\right\}\right).$$

The node representation is then updated:

$$h_i^{(\ell+1)} = \text{UPDATE}^{(\ell)}\left(h_i^{(\ell)}, m_i^{(\ell)}\right).$$

The aggregation function should not depend on the order in which the neighbors are listed. Sum, mean, maximum, and attention-weighted sums are all examples of permutation-invariant aggregation methods.

After one layer, a node can use information from its direct neighbors. After two layers, it can use information from nodes that are two graph steps away. In general, an L-layer model can receive information from an approximately L-hop neighborhood.

This growing receptive field is the main benefit of depth. However, each layer also mixes connected representations. Repeating this process can reduce the differences among nodes and lead to oversmoothing.

3.6 Graph Convolutional Network

The Graph Convolutional Network uses the normalized propagation matrix introduced above (Kipf and Welling 2017). A GCN layer is written as

$$H^{(\ell+1)} = \sigma\left(\widehat{A}H^{(\ell)}W^{(\ell)}\right),$$

where

- $H^{(\ell)}$ contains the node representations at layer ℓ;
- $W^{(\ell)}$ is a trainable weight matrix;
- $\widehat{A}$ performs normalized neighborhood aggregation; and
- σ is an activation function such as ReLU.

The initial matrix is

$$H^{(0)} = X.$$

The multiplication by $\widehat{A}$ mixes each node's representation with the representations of its neighbors. The multiplication by $W^{(\ell)}$ then learns how the aggregated features should be transformed.

GCN is simple and efficient, but repeated multiplication by $\widehat{A}$ has a smoothing effect. As the number of layers increases, the representations can move toward the low-energy space of the normalized Laplacian (Li et al. 2018; Oono and Suzuki 2020).

3.7 GraphSAGE

GraphSAGE separates the representation of the central node from the representation collected from its neighbors (Hamilton et al. 2017). For mean aggregation,

$$m_i^{(\ell)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} h_j^{(\ell)}.$$

The update can be written as

$$h_i^{(\ell+1)} = \sigma\left(W_{\text{self}}^{(\ell)} h_i^{(\ell)} + W_{\text{neigh}}^{(\ell)} m_i^{(\ell)} + b^{(\ell)}\right).$$

Here, the node's own representation and the average neighbor representation have separate trainable transformations.

This differs from GCN, where the self and neighbor contributions are mixed through one normalized propagation operator. GraphSAGE was originally introduced for inductive learning and supports neighbor sampling, although full-graph training is used for Cora in this project. Figure 2 shows the sampling, aggregation, and combination steps in sequence.

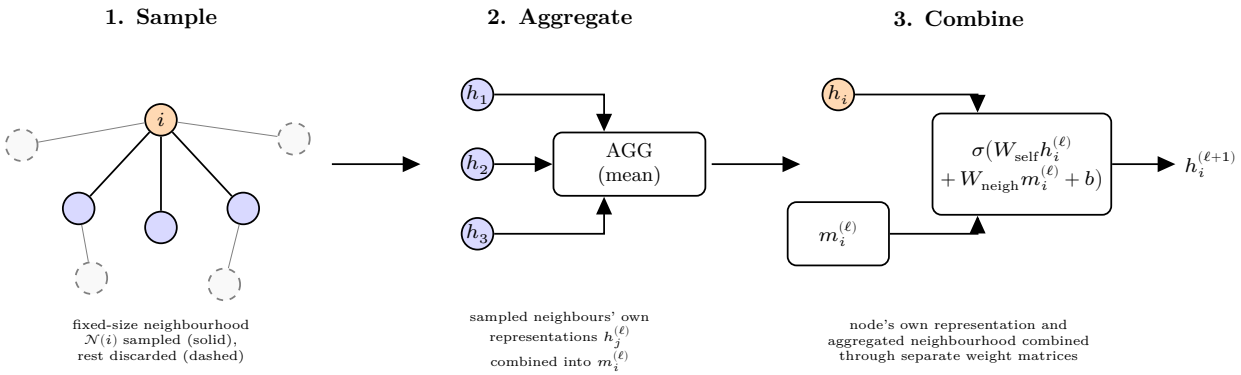


Figure 2: The GraphSAGE sample-and-aggregate pipeline: a fixed-size neighborhood is sampled around each node, the sampled neighbors' own representations are aggregated (mean, here), and the result is combined with the node's own representation through separate weight matrices, recreated as original artwork from the concept in Hamilton, Ying, and Leskovec (2017), Figure 1.

3.8 Graph Attention Network

The Graph Attention Network learns how much importance to assign to each neighbor (Veličković et al. 2018). First, the node representation is linearly transformed:

$$q_i^{(\ell)} = W^{(\ell)} h_i^{(\ell)}.$$

For node i and one of its neighbors j, an unnormalized attention score is

$$e_{ij}^{(\ell)} = \text{LeakyReLU}\left(\left(a^{(\ell)}\right)^T \left[q_i^{(\ell)} \| q_j^{(\ell)}\right]\right),$$

where $\|$ represents concatenation.

The scores are normalized across the neighborhood using softmax:

$$\alpha_{ij}^{(\ell)} = \frac{\exp\left(e_{ij}^{(\ell)}\right)}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp\left(e_{ik}^{(\ell)}\right)}.$$

The new representation is

$$h_i^{(\ell+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{ij}^{(\ell)} q_j^{(\ell)}\right).$$

A larger value of $\alpha_{ij}^{(\ell)}$ gives node j more influence on the new representation of node i.

GAT can use several attention heads. Each head learns its own attention coefficients and transformed features. The head outputs are usually concatenated in hidden layers and may be averaged in the output layer.

Attention makes the aggregation more flexible, but it does not remove the depth problem. A deep GAT still performs repeated neighborhood aggregation and can still produce similar node representations. Figure 3 shows the coefficient computation for a single neighbor alongside the multi-head aggregation.

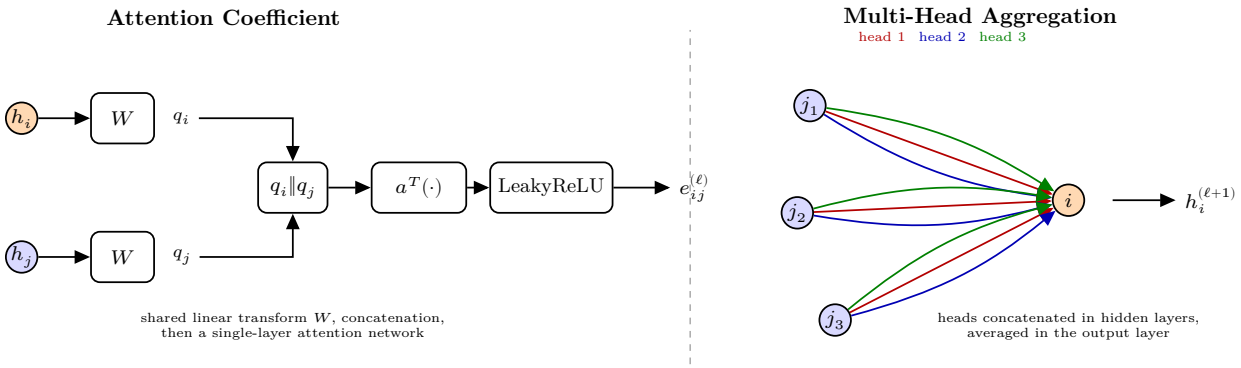


Figure 3: Left, the attention-coefficient computation between a node and one neighbor: a shared linear transform, concatenation, and a single-layer attention network followed by LeakyReLU. Right, multi-head attention aggregation: each head computes its own coefficients, and the head outputs are concatenated in hidden layers or averaged in the output layer. Recreated as original artwork from the concept in Veličković et al. (2018), Figure 1.

3.9 GCNII

GCNII was designed to support deeper graph convolutional networks (M. Chen et al. 2020). It introduces two main ideas: an initial residual connection and an identity mapping.

A simplified GCNII layer is

$$H^{(\ell+1)} = \sigma\left(\left[(1-\alpha_\ell)\widehat{A}H^{(\ell)} + \alpha_\ell H^{(0)}\right]\left[(1-\beta_\ell)I + \beta_\ell W^{(\ell)}\right]\right).$$

The first bracket mixes the propagated hidden representation with the initial representation $H^{(0)}$. This allows the original node information to remain available even in deep layers.

The second bracket combines an identity mapping with a trainable transformation. This limits how strongly each layer can change its input and also provides a more direct path for gradient flow.

GCNII is treated as a separate architecture in this project because its update rule is different from a standard GCN with an added normalization or readout method.

3.10 Mitigation Mechanisms

Three additional methods are applied to the standard architectures in the mitigation experiments.

3.10.1 Residual Connections

A residual connection adds the previous representation to the output of the current layer:

$$H^{(\ell+1)} = F^{(\ell)}\left(H^{(\ell)}\right) + H^{(\ell)}.$$

The direct path helps preserve information from earlier layers and can improve gradient flow. An identity residual can only be applied when the two matrices have the same shape. Figure 4 shows the two routes the representation takes across one layer.

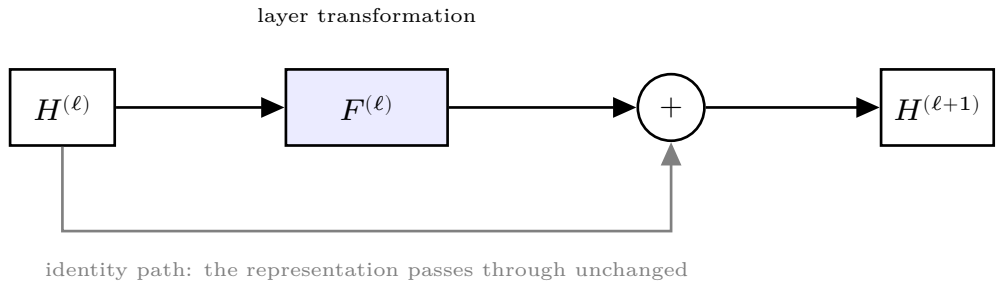


Figure 4: A residual connection around one layer. The representation $H^{(\ell)}$ reaches the addition node by two routes: the layer transformation $F^{(\ell)}$, and an identity path that carries it through unchanged. Their sum forms $H^{(\ell+1)}$, so the earlier representation stays available to the next layer even when the layer transformation contributes little.

3.10.2 PairNorm

PairNorm directly controls the spread of node representations (Zhao and Akoglu 2020). The version used in this project first centers the embeddings:

$$\widetilde{H} = H - \frac{1}{N}\mathbf{1}\mathbf{1}^T H.$$

Each node representation is then normalized separately:

$$h_i^{\mathrm{PN}} = s\frac{\tilde{h}_i}{\|\tilde{h}_i\|_2 + \varepsilon},$$

where s is a scale parameter and ε prevents division by zero. The authors treat s as data-dependent and select it per dataset in their reference implementation, which offers $\{0.1, 1, 10, 50\}$ as candidate values (Zhao 2019).

This is the scale-individual version of PairNorm (PN-SI). It prevents the rows of the embedding matrix from continuously shrinking toward one another. Figure 5 shows the two steps applied to a small set of row vectors.

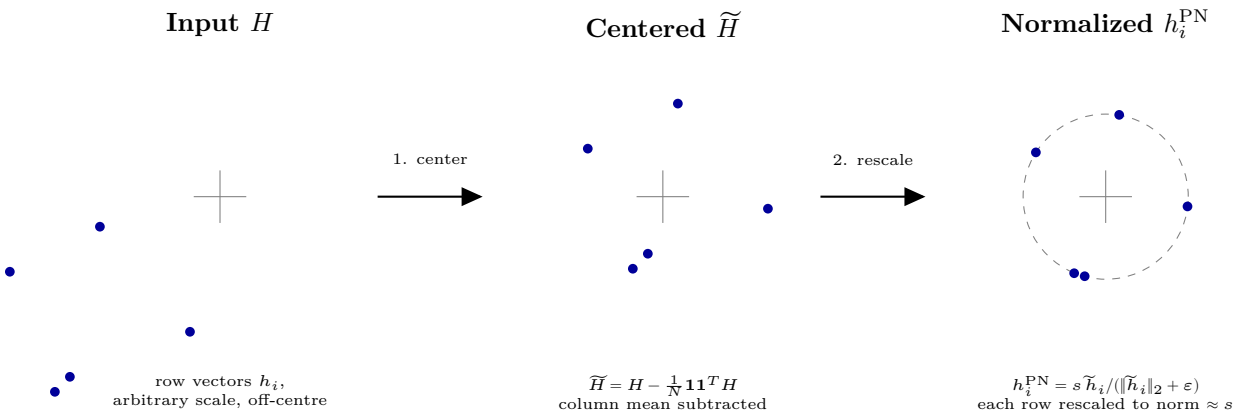


Figure 5: The two-step PairNorm procedure applied to five illustrative row vectors: centering by subtracting the column mean, then per-row rescaling to a fixed norm s. Recreated as original artwork from the concept in Zhao and Akoglu (2020), Figure 2.

3.10.3 Jumping Knowledge

Jumping Knowledge uses representations from several network depths rather than relying only on the final layer (Xu et al. 2018). In this project, the learned hidden representations are combined using element-wise maximum pooling:

$$H_{\mathrm{JK}} = \max\left(H^{(1)}, H^{(2)}, \ldots, H^{(L)}\right),$$

where the maximum is taken separately for each node and hidden feature.

The pooled representation is then passed through a linear output layer. This allows the classifier to use information that appeared in an earlier layer even if later layers have become less useful. Figure 6 shows the per-layer paths into the shared aggregation.

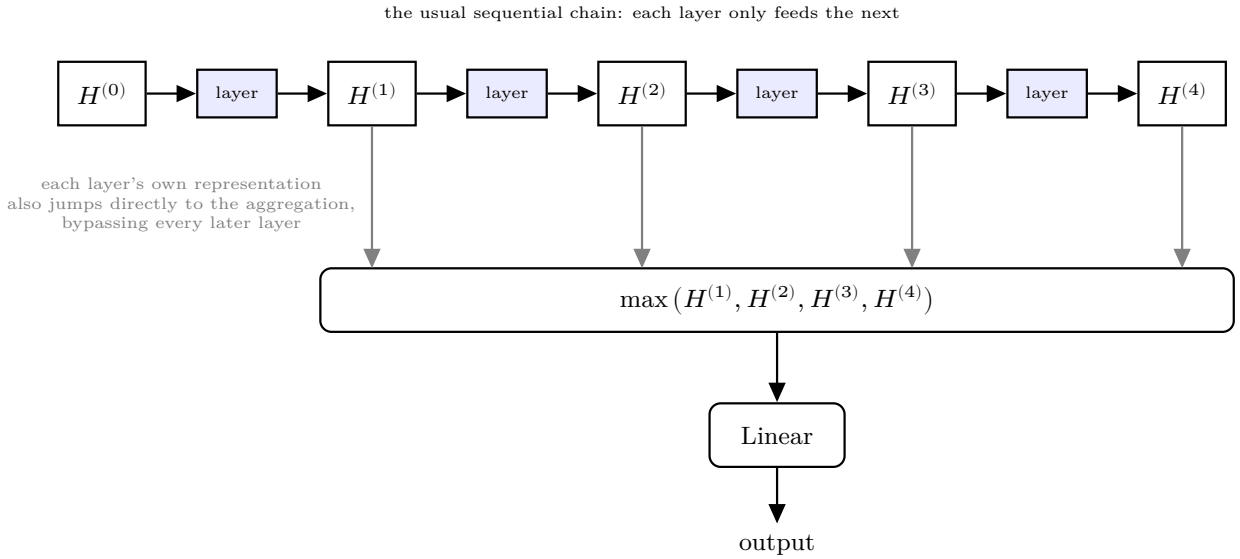


Figure 6: Per-layer representations feeding into a shared aggregation (max-pooling, here) before the final linear readout. Each layer's own representation jumps directly to the aggregation rather than passing only through the next layer, illustrating why the method is called "jumping" knowledge.

3.11 Hidden Representations

For a model with L message-passing layers, the stored representation sequence is

$$H^{(0)}, H^{(1)}, \ldots, H^{(L)}.$$

The matrix $H^{(0)}$ is the original input feature matrix. The later matrices are the learned layer representations.

In this project, a hidden representation is measured after the graph convolution, mitigation hook, and activation function, but before dropout. This represents the deterministic hidden representation passed toward the next layer without including the random dropout mask.

In the implementation these matrices are the list `layerEmbeddings`, indexed the same way, so the prose term and the code identifier name one object.

The purpose of measuring these matrices is to examine how their geometry changes across depth. Oversmoothing is expected to reduce the differences among their rows, but different metrics capture different forms of this change.

3.12 Three Metric Capture Points

The hidden representations are recorded at three stages of each run.

3.12.1 Initialization

The first capture is taken before any gradient update. It describes the behavior of the randomly initialized network.

This capture helps answer whether a deep architecture already contracts its representations before training begins.

3.12.2 Selected Checkpoint

The second capture is taken after the weights with the lowest validation loss have been restored. These are the same weights used to report test accuracy and macro-F1.

This is the most important capture for relating representation behavior to classification performance.

3.12.3 Final Epoch

The third capture is taken using the weights from the final training epoch, before the selected checkpoint is restored.

This capture shows where the optimizer ended. Comparing it with the selected checkpoint reveals whether the model continued to improve, remained unchanged, or deteriorated after its best validation state.

The three captures help separate two situations that could otherwise look similar. A network may start in a collapsed state and fail to train, or it may train and later develop a different representation pattern. A single checkpoint cannot clearly distinguish between these cases.

3.13 Dirichlet Energy

Dirichlet energy measures how much a representation changes over the graph. In this project, the graph used for measurement includes self-loops:

$$\widetilde{G} = (V, \widetilde{E}),$$

with augmented adjacency matrix $\widetilde{A}$ and augmented degrees $\tilde{d}_i$.

For an embedding matrix H, the normalized-Laplacian Dirichlet energy is

$$E(H) = \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \widetilde{A}_{ij} \left\| \frac{h_i}{\sqrt{\tilde{d}_i}} - \frac{h_j}{\sqrt{\tilde{d}_j}} \right\|_2^2 .$$

The same quantity can be written in matrix form as

$$E(H) = \operatorname{Tr}\left(H^T \mathcal{L} H\right).$$

Self-loop terms have zero direct contribution because a node is identical to itself. However, self-loops still change the augmented degrees and therefore change the normalization applied to every edge.

Layers in this project can have different widths. To avoid giving a higher-dimensional layer a larger energy simply because it has more features, the energy is divided by the embedding width:

$$E_{\text{dim}}(H) = \frac{E(H)}{\dim(H)}.$$

A small energy means that the representation changes little across graph edges. A decreasing energy curve is consistent with graph smoothing. However, energy alone does not prove that every node representation is identical, because the zero-energy direction depends on node degree.

3.14 Mean Average Distance

MAD measures the directional separation between node representations (D. Chen et al. 2020). The cosine distance between nodes i and j is

$$D_{ij} = 1 - \frac{h_i^T h_j}{\|h_i\|_2 \|h_j\|_2 + \varepsilon}.$$

A cosine distance close to zero means that the two representations point in nearly the same direction. A larger value means that they are more separated.

For each node, the mean of its nonzero cosine distances is calculated. Let

$$\mathcal{P}_i = \left\{ j : j \neq i \text{ and } D_{ij} > 0 \right\}.$$

The row-level mean is

$$\text{MAD}_i = \frac{1}{|\mathcal{P}_i|} \sum_{j \in \mathcal{P}_i} D_{ij}.$$

The final MAD is the mean over rows that contain at least one nonzero distance:

$$\text{MAD}(H) = \frac{1}{|\mathcal{J}|} \sum_{i \in \mathcal{J}} \text{MAD}_i,$$

where

$$\mathcal{J} = \{ i : |\mathcal{P}_i| > 0 \}.$$

The diagonal is excluded because the distance from a node to itself should not affect the result. If every pairwise distance is zero, MAD is defined as zero rather than left undefined.

MAD and Dirichlet energy are complementary, and the reason is a difference in what each one is invariant to. Cosine distance is invariant to positive rescaling of every node. If $S = \text{diag}(s_1, \ldots, s_N)$ with all $s_i > 0$, then $\text{MAD}(SH) = \text{MAD}(H)$. MAD therefore captures directional alignment but discards embedding magnitude. Dirichlet energy is not scale-invariant; even a global rescaling gives $E_{\text{dim}}(cH) = c^2 E_{\text{dim}}(H)$. It retains both graph-local directional differences and their magnitude.

This distinction makes disagreement between the two informative rather than contradictory. Embeddings can point almost entirely in one direction, producing near-zero MAD, while still carrying substantial Dirichlet energy, because their magnitudes vary across nodes, or because the common direction is not the symmetric Laplacian's degree-weighted null direction. Conversely, global magnitude shrinkage can reduce energy without producing the same directional alignment. Neither metric alone establishes every form of representational collapse.

Chen et al. also define MADGap, the difference between a node's mean distance to remote, multi-hop neighbors and its mean distance to nearby ones, and show that it correlates with test accuracy closely enough to serve as a training-free predictor of model quality (D. Chen et al. 2020). This study does not compute MADGap; only MAD is reported. A metric validated as an accuracy predictor risks blurring the distinction this study is built to keep separate, between a representation having collapsed and a model performing badly, since MADGap's own justification is that the two track each other. Reporting MAD alone keeps the representation-level measurement and the classification result independent, so that either can be read without presupposing the other.

3.15 Comparable Hidden-Layer Band

Not every stored representation should be compared in the same layer-wise curve.

The input matrix $H^{(0)}$ is excluded because it contains sparse 1,433-dimensional bag-of-words features rather than learned hidden representations. A direct comparison between this input and a 64-dimensional hidden layer would mix changes in representation type, width, and scale with the effect of message passing.

For models using the standard last-layer readout, the final representation is also excluded. It contains seven-dimensional class logits rather than hidden features. Cross-entropy training directly shapes this decision space, so its energy measures a mixture of classification behavior and graph smoothing.

The comparable band is therefore formed from the contiguous learned hidden layers that share the model's main hidden width.

For a standard last-layer readout, the band is

$$\mathcal{B} = \{1, 2, \dots, L-1\}.$$

For Jumping Knowledge, the final convolution also produces a hidden-width representation before the separate readout. Its comparable band is therefore

$$\mathcal{B}_{\mathrm{JK}} = \{1, 2, \dots, L\}.$$

Defining this band from the actual tensor widths prevents different architectures from being compared using representation spaces that do not have the same meaning.

3.16 Relative Energy Curve

Raw energy values can differ because of weight initialization, architecture, and representation magnitude. To focus on the change across layers, energy is reported relative to the first layer in the comparable band.

Let

$$\ell_0 = \min(\mathcal{B}).$$

The relative energy at layer ℓ is

$$R_\ell = \frac{E_{\dim}\left(H^{(\ell)}\right)}{E_{\dim}\left(H^{(\ell_0)}\right)}.$$

Therefore,

$$R_{\ell_0} = 1.$$

A value below one means that the energy has decreased relative to the first comparable hidden layer. A value above one means that the energy has increased.

This ratio is useful for comparing the shapes of energy curves across models, but it should not be interpreted as a complete measure of embedding collapse. The MAD curve and the original energy values are also needed.

3.17 Contraction Slope

A single summary value is obtained by fitting a line to log-energy across the comparable band. For each $\ell \in \mathcal{B}$,

$$\log\left(\max\left\{E_{\dim}\left(H^{(\ell)}\right), \varepsilon_E\right\}\right) \approx a + b\ell,$$

where

$$\varepsilon_E = 10^{-12}.$$

The coefficient b is called the contraction slope.

- A negative slope means that energy generally decreases with depth.
- A slope near zero means that energy remains relatively stable.
- A positive slope means that energy generally grows across the measured layers.

The energy floor prevents the logarithm of zero. Its consequence is that very strong contraction or growth can appear less extreme after values reach the floor. The floor can pull the estimated slope toward zero, but it cannot create a trend that was not already present.

At least two comparable layers are required to fit a slope. A two-layer model with a standard last-layer readout has only one layer in its comparable band. Its contraction slope is therefore undefined rather than equal to zero.

The slope summarizes a curve, but it does not replace the curve. Two models can have similar fitted slopes while showing different behavior in their early and late layers.

3.18 Connection to Oversmoothing Theory

Oversmoothing names the mechanism: repeated propagation contracting representations toward one another. Collapse names what the metrics register when it happens, MAD falling toward zero as directions align and Dirichlet energy falling as neighboring representations converge. The two are used in that sense throughout: oversmoothing is a claim about cause, collapse a claim about what was measured.

A common theoretical view of oversmoothing describes repeated graph propagation as a contraction toward the low-frequency space of the graph Laplacian. Under simplifying assumptions, the layer energies can satisfy a bound of the form

$$E\left(H^{(\ell+1)}\right) \leq (1-\lambda)^2 \left\|W^{(\ell)}\right\|_2^2 E\left(H^{(\ell)}\right),$$

where λ represents a graph spectral quantity and $\|W^{(\ell)}\|_2$ is the spectral norm of the layer weight matrix (Cai and Wang 2020).

The bound explains two important points.

First, the graph operator can contract the representation from one layer to the next. Repeating this contraction can produce an approximately exponential decrease in energy.

Second, the layer weights also affect the result. Large weight norms may reduce, cancel, or reverse the contraction caused by graph propagation. For this reason, energy growth in a trained model is not automatically inconsistent with oversmoothing theory. The trainable transformations are part of the mechanism, not a separate source of noise.

The experiments do not test the numerical tightness of this bound because the spectral gap and layer weight norms are not recorded across the full sweep. The bound is used to motivate the log-energy analysis and the interpretation of the contraction slope.

3.19 Classification Metrics

3.19.1 Accuracy

Test accuracy is the proportion of test nodes classified correctly:

$$\text{Accuracy} = \frac{1}{|V_{\text{test}}|} \sum_{i \in V_{\text{test}}} \mathbf{1}\left(\hat{y}_i = y_i\right).$$

Accuracy gives equal weight to each test node.

3.19.2 Precision and Recall

For class c, precision is

$$P_c = \frac{TP_c}{TP_c + FP_c},$$

and recall is

$$R_c = \frac{TP_c}{TP_c + FN_c}.$$

Here, TP_c, FP_c, and FN_c are the true positives, false positives, and false negatives for class c.

The class-specific F1 score is

$$F_{1,c} = \frac{2P_cR_c}{P_c + R_c}.$$

3.19.3 Micro-F1

Micro-F1 first combines the counts from all classes:

$$F_{1,\text{micro}} = \frac{2P_{\text{micro}}R_{\text{micro}}}{P_{\text{micro}} + R_{\text{micro}}}.$$

For single-label multiclass classification, micro-F1 is mathematically equal to accuracy. Every incorrect prediction creates one false positive for the predicted class and one false negative for the correct class. Micro-F1 is still reported because it is part of the project requirements.

3.19.4 Macro-F1

Macro-F1 averages the class-specific F1 scores:

$$F_{1,\text{macro}} = \frac{1}{C}\sum_{c=1}^{C} F_{1,c}.$$

Each class contributes equally, regardless of how many test examples it contains. Macro-F1 is therefore useful for identifying models that perform reasonably in overall accuracy but concentrate most of their predictions into only a small number of classes.

3.19.5 Reference Baselines

Two reference values give the accuracy and macro-F1 numbers reported in Section 6 a floor to compare against. The uniform-random rate is the accuracy a classifier achieves by guessing among the C classes with equal probability and no information about the input:

$$\text{Accuracy}_{\text{random}} = \frac{1}{C}.$$

For Cora's seven classes, this is approximately 14.3%.

The majority-class rate is the accuracy achieved by a fixed classifier that always predicts whichever class is most frequent in the test split, without using any node feature or graph information. Unlike the uniform-random rate, this floor depends on the actual class distribution of the test split rather than on the number of classes alone; on the standard split used throughout this study, the majority class accounts for 319 of the 1,000 test nodes, a majority-class floor of 31.9%. A trained model

whose test accuracy falls below this value is performing worse than a rule that ignores the input entirely, which is a stronger and more specific claim than a low accuracy number on its own.

The same reasoning gives a floor for the training loss. A classifier that outputs a uniform distribution over the seven classes, carrying no information about the input, has cross-entropy $\ln 7 = 1.9459$. A training loss sitting at that value means the model has not moved off the uninformed prediction, so the figure is what makes a small loss movement legible as no movement at all (Section 6.5).

A third set of reference values comes from the published depth-2 Cora accuracies each architecture's own paper reports, against which Section 6.1 checks this study's implementation. GCN is reported at 81.5% (Kipf and Welling 2017); the closer comparator for this study's width is the 64-feature GCN variant reported at 81.4% (± 0.5) in Veličković et al.'s own comparison table (Veličković et al. 2018). GAT is reported at 83.0% (± 0.7) over 100 runs on the same 140/500/1000 split (Veličković et al. 2018). GraphSAGE has no published Cora figure to compare against: Hamilton et al. evaluate on Web of Science, Reddit, and PPI, all inductive settings (Hamilton et al. 2017).

3.20 The Cora Citation Network

Cora is a standard benchmark for semi-supervised node classification (Sen et al. 2008; Yang et al. 2016). It contains

- 2,708 scientific papers;
- 1,433 binary input features per paper; and
- seven research-topic classes.

The seven classes are:

1. Case-Based Reasoning;
2. Genetic Algorithms;
3. Neural Networks;
4. Probabilistic Methods;
5. Reinforcement Learning;
6. Rule Learning; and
7. Theory.

Each feature indicates whether a vocabulary term appears in a paper. The original feature vectors are sparse because each paper contains only a small part of the full vocabulary.

The graph connects papers through citation relationships. The processed Planetoid version used in this project treats these connections as undirected for message passing.

The public split contains:

Table 1: The standard Planetoid public split of Cora.

Split	Number of nodes	Purpose
Training	140	Used to calculate the loss and update the model
Validation	500	Used for checkpoint selection and early stopping
Testing	1,000	Used only for final evaluation

The 140 training nodes in Table 1 contain 20 labeled papers from each of the seven classes. The

remaining nodes still participate in message passing even though their labels are not used in the training loss.

The graph's degree distribution is highly skewed: node degree in the augmented graph ranges from 2 to 169, with a mean of approximately 4.9. This skew is not a decorative statistic. It is the reason the augmented normalized Laplacian's null space diverges from the constant vector on Cora, discussed above: on a regular graph, where every node has the same degree, the square-root-of-degree direction and the constant direction coincide, but Cora's wide degree range keeps them apart, and Section 6 depends on this distinction directly.

Cora also exhibits homophily: papers tend to cite other papers on related topics, so a node's label is correlated with its neighbors' labels more often than chance alone would predict. This is what makes neighborhood aggregation an informative operation for this task in the first place; on a graph without homophily, combining a node's representation with its neighbors' would not, on average, help classify it.

3.21 Feature Normalization

The feature vectors are row-normalized before training. For node i,

$$\tilde{x}_i = \frac{x_i}{\sum_{k=1}^{d} x_{ik} + \varepsilon}.$$

After normalization, the nonzero features in each row sum to approximately one. This prevents papers containing more vocabulary terms from automatically having feature vectors with much larger total magnitudes.

The same normalization is used for every architecture and depth. It is therefore a fixed part of the data pipeline rather than an experimental variable.

3.22 Graph Used by the Models and Metrics

The dataset loader returns the undirected graph without self-loops. The GNN layers add their required self-loops and normalization internally.

The oversmoothing metrics build a separate augmented copy of the graph:

$$\widetilde{A} = A + I.$$

The same augmented metric graph is used for GCN, GraphSAGE, GAT, and GCNII. This is important for a cross-architecture comparison. If each architecture were measured using a different graph operator, a difference in energy could come from either the embeddings or the measuring graph.

For GAT, this means that Dirichlet energy is measured relative to the fixed Cora graph rather than the model's learned attention-weighted graph. The result describes how GAT's embeddings vary over Cora's original structure. It does not describe variation over a different attention graph at every layer.

3.23 Summary

The project treats a GNN layer as both a feature transformation and a graph propagation step. Increasing the number of layers expands the receptive field, but it may also change the node representations in ways that reduce their usefulness for classification.

GCN, GraphSAGE, GAT, and GCNII use different update rules, so they are not expected to respond to depth in exactly the same way. Residual connections, PairNorm, and Jumping Knowledge also address different parts of the problem.

Classification accuracy and macro-F1 show whether the predictions remain useful. Dirichlet energy and MAD show how the hidden representation changes. The three capture points separate initialization behavior from behavior that develops during training. Finally, the comparable band and contraction slope allow the layer-wise energy curves to be compared without mixing raw inputs, hidden representations, and output logits.

4 Implementation Details

This section describes what was built and why, at the level of design decisions rather than line by line; Section 5 describes how the code expresses those decisions.

4.1 Setup

`data.LoadCora` loads the public-split Cora citation network through PyTorch Geometric's `Planetoid` dataset (Fey and Lenssen 2019) and row-normalizes the node features (Section 3.21), matching Kipf and Welling's published GCN setup (2017, sec. 5.2). An invariant check confirms the loaded graph's shapes and raises, rather than warns, if any fails, so a PyG version upgrade that silently altered the graph's shape is caught immediately. Every run is seeded on the integer list 0 through 9, with the same seed controlling both the model's random initialization and the training stochasticity that follows, so a reported mean and standard deviation over ten seeds reflects ten independent draws rather than ten runs sharing one initialization. Every run also records its own environment at write time; all 534 records carry Python 3.14.4, PyTorch 2.13.0 (CPU build), PyTorch Geometric 2.8.0, and device `cpu`.

4.2 Model Implementation

Section 3 gives four update rules. The implementation realizes all four as one parameterized model, in which `convType` selects the convolution and everything else (depth, width, mitigations, and the point at which each layer's representation is recorded) is identical across architectures.

Mitigations attach to the shared layer loop by composition rather than inheritance, so the unmitigated baseline and every mitigated variant execute the identical control-flow path and a measured difference between them is attributable to the mitigation itself (Section 5.3's `models` entry gives the full reasoning).

Hidden width is 64 for every architecture, rather than each architecture's own published width (16 for GCN; 8 heads of 8 for GAT). Capacity parity is the precondition for comparing architectures at matched depth: at width 16, GAT's eight heads would each attend over two-dimensional keys, close to degenerate, placing a confound in the study's headline figure that has nothing to do with depth. The fidelity arm (Section 6.1) recovers the published width as a separately measured quantity.

4.2.1 Keeping Depth Comparable Across Architectures

Table 2 records two numerical facts that distinguish GCNII's per-layer computation from the other three architectures'.

Table 2: Where GCNII's per-layer computation departs from GCN, GraphSAGE, and GAT.

Architecture	Uses $H^{(0)}$ at every layer	Extra layers outside counted depth L
GCN	No	None
GraphSAGE	No	None
GAT	No	None
GCNII	Yes	2 (input projection, output projection)

Depth is defined as a message-passing hop count, and GCNII is made to match that definition exactly. Its two required linear projections sit outside the counted depth L, so GCNII's L layers are L real message-passing hops. Depth is the study's primary comparison axis, and this is what makes GCNII's L the same quantity as the others'. The known cost: those two projections give GCNII two extra parameterized layers the other three architectures do not have, bounded in size and not scaling with depth, which can only overstate GCNII's capacity relative to its peers, at every depth (revisited in Section 6.6).

4.2.2 Recording the Hidden Representation

Where the hidden representation is recorded within a layer follows Section 3.11; Figure 7 places that position inside the layer. Three figures in Section 6 are computed from the recorded tensors: Figure 11, Figure 10, and Figure 14.

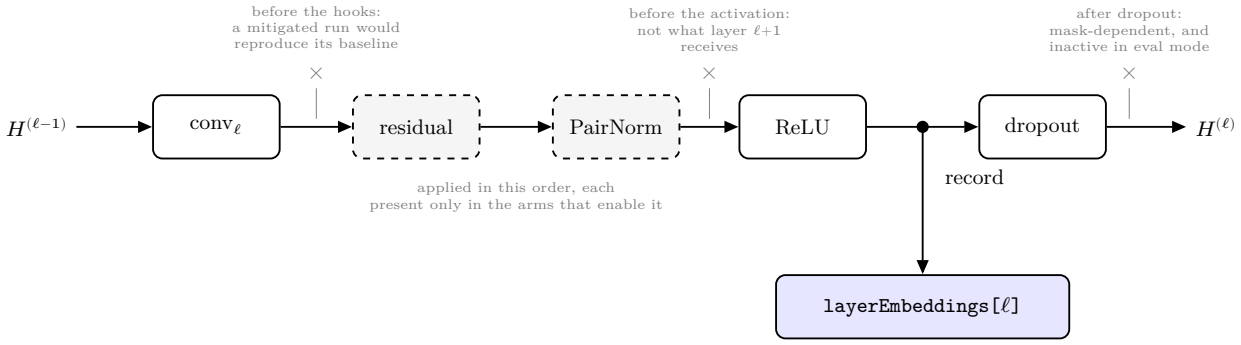


Figure 7: One layer's interior and the point at which its representation is recorded. The recording point sits after the convolution, after every mitigation hook, and after the activation, but before dropout. The three crossed positions are the alternatives this subsection rejects, each labeled with the reason. Hook boxes are dashed because a hook is present only in the arms that enable it; their order is fixed at residual before PairNorm regardless of the order in which the mitigation names are supplied.

- **Before dropout**, because inverted dropout rescales surviving units by $1/(1-p)$ at train time, which would inflate raw Dirichlet energy by a constant that the energy ratio absorbs

but that MAD does not respond to identically, making the two metrics disagree for a reason that is an artifact rather than a finding.

4.3 Mitigation Implementations

The three mechanisms specified in Section 3.10 (residual connections, PairNorm, and Jumping Knowledge) attach to the base model independently of which architecture is running. Our proposal listed GCNII as a fourth mitigation; we implement it instead as a fourth convolution type, since it modifies the layer's update rule rather than wrapping an existing one. Table 3 states what each mechanism does to the representation and how each is implemented.

Table 3: The three mitigation mechanisms, their effect on the representation, and the implementation choice made for each.

Mechanism	What it does to the representation	Learned parameters	Implementation choice
Residual	Adds the previous layer's output back into the current layer's output, so each layer starts from what came before instead of replacing it	None	Plain identity addition on interior layers; skipped where input and output widths differ
PairNorm	After each layer, recenters all node representations and rescales each to a fixed length, holding their total spread constant as depth grows	None	PN-SI variant, scale fixed at 1.0, untuned
Jumping Knowledge	Keeps every layer's representation and combines them at the readout, so the final prediction is not forced to rely on the deepest layer alone	One linear readout	Element-wise max pooling, one of the three aggregations in Xu et al. (2018)

Hook ordering is fixed at residual before PairNorm, regardless of the order the mitigation names are supplied in. Residual is part of the update rule and defines what the layer's output is; PairNorm then controls the spread of that output (Section 3.10), which requires it to act last.

4.3.1 Residual

Residual is implemented as identity addition, following the depth study in Kipf and Welling's Appendix B (Kipf and Welling 2017). The mechanism of Section 3.10 adds a layer's input to its output, which requires both to carry the same number of features per node. Layer 1 maps 1433 input features onto the hidden width of 64, and under the standard last-layer readout the last layer maps 64 onto the seven classes, so the addition is skipped at both boundaries and fires on interior layers alone.

4.3.2 PairNorm

PairNorm is applied after each convolution, and in this study only to GCN. We use PN-SI (Section 3.10), which the authors' usage guidance pairs with GCN: their repository reports that plain PairNorm behaves badly under the symmetric normalized adjacency GCN uses (Zhao 2019). A later note in the same repository traces part of that behavior to a bug and suggests plain PairNorm may

now suffice, while stating this has not been fully tested, so we retain PN-SI. We hold s at 1.0 rather than selecting it per dataset, because the coordinated search (Section 4.6) holds one configuration fixed across every architecture and depth; tuning PairNorm's scale alone would compare a tuned mitigation against untuned baselines. Whatever PairNorm scores at this setting is therefore a floor: tuning s could raise it, not lower it.

4.3.3 Jumping Knowledge

Jumping Knowledge aggregates the stored hidden representations by element-wise max pooling and projects the result to the output width with a single linear layer. Max pooling holds the readout's input width at the hidden width regardless of depth; concatenation, one of the three aggregations in Xu et al. (2018), would make it $T \times$ hiddenDim for T pooled layers, so the JK arm's parameter count would vary along the study's comparison axis (Section 4.2.1). JK's comparable band, $1..L$ rather than $1..L-1$, follows directly from the band-derivation rule in Section 3.15.

4.4 Training and Evaluation Harness

4.4.1 Optimizer and Weight Decay

Every run uses PyTorch's Adam implementation (`torch.optim.Adam`) (Paszke et al. 2019) at the configuration the search in Section 4.6 selected (`learningRate=0.01`, `dropout=0.5`, `weightDecay=5e-4`), the same values Kipf and Welling publish. Weight decay is applied uniformly across every layer's parameters, which keeps the regularization from becoming an implicit function of depth (Section 4.2.1).

4.4.2 Early Stopping and Checkpoint Selection

Early stopping and checkpoint selection are both driven by validation loss. Training halts when validation loss has not improved for `patience` consecutive epochs; the reported and captured checkpoint is the epoch of minimum validation loss (Section 3.12.2), whose weights are restored after the loop ends. Kipf and Welling specify a stopping rule (200 epochs maximum, halting if validation loss does not decrease within a window of 10) (Kipf and Welling 2017), and this study adds the selection rule, tying it to the same signal so `bestEpoch` is unambiguous. Validation loss and validation accuracy are recorded at every epoch in `trainingCurve`.

4.4.3 Patience

Patience is 100 epochs, held constant across every architecture, depth, mitigation arm, and seed, with `maxEpochs=1000` as a ceiling. A 32-layer network may sit on a plateau for many epochs before its loss begins to descend, or may never descend. At Kipf and Welling's window of 10, a run whose validation loss stops improving after the first epoch halts near epoch 12 (Section 6.4), leaving a flat training curve that a non-converging optimization and a genuinely untrainable architecture would share. Holding patience constant across depth keeps the depth comparison valid (Section 4.2.1).

4.4.4 Three Metric Captures

Three metric captures are taken per run, each an explicit eval-mode forward pass under `torch.no_grad()` (Section 3.12): epoch 0, before any gradient step; the final epoch, on the weights the loop ended with; and the selected checkpoint, after the best `state_dict` is restored. The order matters, and Figure 8 places the three on the sequence of one run. The final-epoch capture is

taken while the loop's own weights are still loaded, before the checkpoint weights are restored. Taken afterward, both captures would read the same weights, and `finalMetrics` would duplicate `checkpointMetrics` on every run. What the two record separately is how the model changed over the epochs between its best validation loss and the end of the loop (Section 6.4).

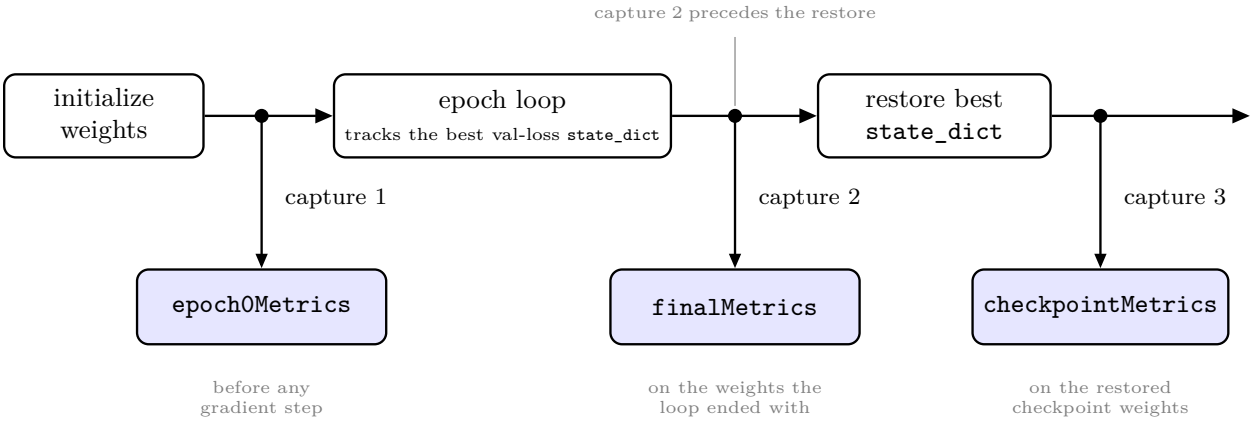


Figure 8: The order of the three metric captures within one run. Each is a separate eval-mode forward pass. The capture on the final epoch's weights is taken before the best-validation `state_dict` is restored.

4.5 Metrics Implementation

Each choice below holds the measurement instrument fixed against something the study varies: architecture, split, layer width, or depth.

4.5.1 The Fixed Metric Graph

Dirichlet energy and MAD are computed on the same graph for all three architectures: the augmented graph $\widetilde{G}$ of Section 3.22. The three aggregate over their neighbors differently, GCN by a fixed symmetric normalization and GAT by attention weights that are learned and change at every layer and every epoch, and the metric graph does not follow those differences.

The metric graph is a measuring instrument rather than part of the model under test, and holding it fixed makes the embeddings the only thing that varies across the cross-architecture comparison. Matching it to GAT's own propagation graph is ill-defined in the first place, since GAT's attention re-weights the graph per layer and throughout training. Energy at consecutive layers would then be measured against different graphs, making the fitted contraction slope compare quantities with different units. A model could also lower measured energy purely by concentrating attention with representations otherwise unchanged, so the metric would partly measure itself. For GAT, the fitted decay rate describes GAT's embeddings with respect to Cora's fixed graph structure, not with respect to GAT's own attention-weighted propagation, legitimate and necessary for a cross-architecture comparison, but not the exact quantity Cai and Wang's bound (2020) describes for an attention-based architecture.

4.5.2 Dirichlet Energy and MAD

Both metrics are computed over all 2,708 nodes; neither consumes labels, so the train, validation, and test masks of Section 3.20 play no part.

Dirichlet energy is per-dimension, $E(H)$ divided by the layer's width, which removes the effect of comparing a 1433-dimensional input against a 64-dimensional hidden representation, and is computed as Section 3.13 defines it. Self-loop terms contribute zero to the energy sum but change every node's augmented degree, which is why the energy is computed on the augmented graph (Section 3.2).

MAD follows the reduction convention of Eq. 1-4 in D. Chen et al. (2020) (Section 3.14): a cosine-distance matrix, averaged over non-zero entries only, at both the per-row stage and the final stage. Two further details:

- The diagonal is excluded before reduction. For a row with positive norm this changes nothing: a row's self-cosine-similarity is 1, so it already contributes a zero distance that the non-zero filter drops. It matters for an all-zero row, where the row-norm clamp (Section 5.3) makes the self-similarity compute as 0 rather than 1, leaving a spurious non-zero self-distance that would survive the filter.
- Both reduction denominators return 0 rather than `NaN` when a count is 0, the case a fully collapsed configuration produces.

4.5.3 Contraction Slope and Relative Energy

The contraction slope of Section 3.17 is computed once per capture, by least squares on `log(energy)` against layer index over the comparable band of Section 3.15. Energies are floored at $\varepsilon_E = 10^{-12}$ before the fit, since collapsed layers reach zero energy in float32 and the logarithm of zero is undefined.

For the unmitigated depth-32 GCN baseline (`gcn_none_d32_s0`) the floor binds on 6 of 31 band layers at epoch 0, where the reported slope is `-0.7579` against an unfloored `-0.8508`, and on 22 of 31 at the checkpoint, where it is `+0.8622` against `+1.1435`. The floor pulls a reported slope toward zero whichever direction it runs, so both reported values understate the magnitude of change.

Where the band has fewer than two points the fit returns `nan`, following Section 3.17. At depth 2 under the standard last-layer readout the band is a single index, so every depth-2 configuration outside the Jumping Knowledge arm carries no slope; JK's band of $1..L$ admits a fit at that depth.

The relative energy curve of Section 3.16 divides each band layer's energy by the energy at $\ell_0 = \min(\mathcal{B})$, the first layer in the band. Section 6 reports its endpoint, `E_last/E_1`.

Values that come out `NaN` or infinite, a zero reference energy for instance, are plotted untouched rather than dropped, so a collapsed run shows a visible gap in a curve rather than a fabricated point.

The energy curve is divided by layer width only; the squared Frobenius norm $\|H_l\|_F^2$ is recorded per layer, so a magnitude-normalized variant remains derivable at aggregation time. Cai and Wang's bound, $E_{l+1} \leq (1-\lambda)^2 \|W\|^2 E_l$, places weight-norm growth inside the mechanism the bound describes.

4.6 The Experiment Grid

The sweep is organized into six *arms*, in the clinical-trial sense: one control condition, four that vary a single factor against it, and a hyperparameter search. Together they total 534 runs (Table 4).

Table 4: The six sweep arms, their configurations, and their run counts.

Arm	Configuration	Depths	Seeds	Runs
A	Unmitigated sweep: `{gcn, sage, gat}`	2-32	10	150
B	Mitigation ablation on GCN: `{residual}`, `{pairnorm}`, `{jk}`, `{pairnorm, residual}`	2-32	10	200
C	GCNII	2-32	10	50
D	Arm B's most effective mitigation, carried to `{sage, gat}`	2-32	10	100
E	Fidelity: GCN, `hiddenDim=16`	2	10	10
F	Hyperparameter search: GCN, `learningRate` x `dropout` x `weightDecay`	2	3	24
				534

Arm D is generated only after arm B is aggregated, its mitigation selected from arm B's outcome (Section 6.7); arm F's grid is `learningRate` in `{0.005, 0.01}`, `dropout` in `{0.5, 0.6}`, and `weightDecay` in `{5e-4, 1e-3}`.

The depth grid `{2, 4, 8, 16, 32}` is log-spaced because Cai and Wang's contraction bound (Cai and Wang 2020) predicts exponential energy decay in depth: log-spaced depths are evenly spaced on the log-energy axis the energy figures use. A candidate additional depth of 24 was rejected, since it buys resolution only where the curve has already collapsed and breaks that spacing.

The hyperparameters are tuned once, in arm F: GCN at depth 2, over the eight candidate configurations. The selected values are then frozen across every architecture and depth in arms A through E. Tuning per depth or per architecture would confound the depth effect the study measures with a hyperparameter effect. Section 6.1 reports the outcome, and its limitation: three seeds do not cleanly separate the eight candidates.

5 Explanation of the Source Code

This section walks through how the code expresses the design of Section 4, at the per-module level: the key classes and functions in each module, and the data flowing between them. A reader with the repository open can follow along.

5.1 Repository Map and Reading Order

5.1.1 Dependency Order

Table 5 gives the module dependency order, which is also the build order and the order to read the modules in; Figure 15 in the appendix draws the same structure as a class diagram.

Table 5: Module dependency order, which is also the build order and the reading order.

Order	Module	Depends on	Notes
1	`data`	(none)	Root of the dependency graph
2	`models`	`data`, field names only	Declares the two protocols `mitigations` implements, without importing `mitigations`

Order	Module	Depends on	Notes
3	`metrics`	`data`; `models`, output shape only	A fixed instrument, not configuration-aware
4	`train`	`models`, `metrics`	Owns the only optimizer in the repository
5	`mitigations`	The two protocols `models` declares	Sits after `train` so the null-object default lets the harness run and the baselines reproduce before any mitigation exists
6	`experiments`	Every other module	The only module that writes to disk
7	`viz`	`data`, labels only; the records `experiments` writes	The only module whose output a reader sees directly

5.1.2 Entry Points

Three entry points sit outside this module graph:

- `src/run_sweep.py` executes the sweep against the six arms, in the order F, A, C, E, B, D. Arms F and B each precede an aggregation step whose outcome another arm needs, so B runs after C and E, and D runs last.
- `src/generate_report_figures.py` regenerates every figure and table from `results/` alone.
- The pytest suite under `src/tests/` is run directly, rather than through either driver.

5.2 Shared Interfaces in Code

5.2.1 The Model's Return Type

`GnnModel.Forward(x, edgeIndex) -> tuple[Tensor, list[Tensor]]` (`src/models/gnn_model.py`) is the one place this interface is expressed in code: it returns `(logits, layerEmbeddings)`, with `layerEmbeddings[0]` always the raw input tensor `x` and every subsequent entry the corresponding layer's post-activation, pre-dropout output.

5.2.2 The Results Record

The results record is assembled entirely inside `train.TrainRun` (`src/train/harness.py`) and written by exactly one caller, `experiments.RunSweep` (`src/experiments/runner.py`), via `_AtomicWriteJson`. `train` never touches the filesystem; `TrainRun` returns a plain `dict`, and a test can call it directly against an in-memory config with no filesystem fixture. The filename convention lives in one place, `experiments.runner.ResultPath`.

5.2.3 Runtime Assertions

Two runtime assertions enforce invariants the type system cannot: in `experiments.runner.RunOne`, `("jk" in config.mitigations) == (config.readout == "jk")` is checked before training begins, so a `RunConfig` in which the mitigation list and the readout disagree about Jumping Knowledge fails immediately rather than producing a record with an internally inconsistent `config` block; and in `metrics.oversmoothing.BuildAugmentedOperator`, the incoming `edgeIndex` is checked for self-loops and the constructor raises if any are found, which would otherwise double-augment every node's degree.

5.2.4 Attached Embeddings

`layerEmbeddings` is returned attached to the autograd graph, and detaching happens only at the three capture sites in `train`, never inside `models`. Under Jumping Knowledge the readout consumes the whole embedding stack to produce the logits, so the training loss backpropagates through every entry in the list; detaching inside `Forward` would silently zero the gradient to every layer except the last, and JK would train incorrectly with no error raised anywhere.

5.3 Per-Module Walkthroughs

Table 6: What each module owns, and what it deliberately does not.

Module	File(s)	Owns	Does not own
`data`	`cora.py`	`LoadCora`, `AssertGraphInvariants`	Self-loop insertion, degree normalization
`models`	`gnn_model.py`, `architectures.py`, `protocols.py`	The layer loop, the four architecture subclasses	Mitigation logic (declares an interface for it only)
`metrics`	`oversmoothing.py`	`DirichletEnergy`, `MeanAverageDistance`, `SelectComparableBand`, `FitContractionSlope`	Training, mitigations, experiment configuration
`train`	`harness.py`	The optimizer, the training loop, result-record assembly	The filename convention, any file I/O
`mitigations`	`hooks.py`, `readouts.py`, `naming.py`	Three mechanism implementations, one naming helper	`GnnModel`, `train`, `experiments`
`experiments`	`grid.py`, `runner.py`	The declarative grid, model construction from a `RunConfig`, all filesystem writes	Any numerical computation
`viz`	`aggregation.py`, `figures.py`, `tables.py`	Loading, tidying, aggregating, plotting, table export	Model construction

What follows for each module is the reasoning Table 6 cannot carry: the non-obvious implementation choices and why each one exists.

5.3.1 `data`

Every invariant check (`num_nodes`, feature shape, class count, undirectedness, self-loop absence, mask sizes, pairwise mask disjointness) is a single vectorized tensor reduction (`.sum()`, boolean `&`, PyG's `contains_self_loops`). `AssertGraphInvariants` raises `ValueError` explicitly rather than using a bare `assert`, so the checks cannot be compiled out under `python -O`.

5.3.2 `models`

Hooks and the readout are injected at model construction, as a list of `LayerHook` objects plus one `Readout` object, and `models` imports nothing from `mitigations`; both sides depend on two protocols declared inside `models` (Figure 15 gives both protocols' exact signatures). The unmitigated baseline is constructed with `layerHooks = []` and `readout = LastLayerReadout()`, so the layer loop contains no branch on whether mitigations exist.

`GnnModel.__init__` builds `self.convs` as an `nn.ModuleList` by calling the subclass-supplied `BuildLayerConv` once per layer, computing each layer's input and output width from `numLayers`, `hiddenDim`, and whether the current layer is the final, logit-emitting one. `Forward` itself, reproduced below, is where the recording point from Section 4.2 is visible in code:

```
def Forward(self, x: Tensor, edgeIndex: Tensor) -> tuple[Tensor, list[Tensor]]:
    """Returns (logits [N, C], layerEmbeddings)."""
    h = x
    x0 = x
    layerEmbeddings: list[Tensor] = [x]
    for l, conv in enumerate(self.convs, start=1):
        hPrev = h
        h = conv(h, edgeIndex, x0)
        for hook in self.layerHooks:
            h = hook.Apply(h, hPrev, edgeIndex)
        isFinalAndLogits = (l == self.numLayers) and self.readout.FinalLayerIsLogits
        if not isFinalAndLogits:
            h = torch.relu(h)
        # tapping after activation, before dropout: the representation the
        # next layer actually receives, independent of the stochastic
        # dropout mask
        layerEmbeddings.append(h)
        if not isFinalAndLogits:
            h = self.dropoutLayer(h)
    logits = self.readout.Apply(layerEmbeddings)
    return logits, layerEmbeddings
```

(`src/models/gnn_model.py`, `GnnModel.Forward`, reproduced verbatim except for the outer method indentation.)

`GcniiModel.Forward` (in `architectures.py`) duplicates this loop rather than reusing the base class's, since its input and output projections do not fit the base class's per-layer construction: this is the one place in `models` where the same shape of loop is written twice, a direct consequence of `GCN2Conv`'s equal-width constraint. The non-obvious point worth a reader's attention: `_GatConvAdapter` divides the requested total output width by the head count before constructing `GATConv`, so that `hiddenDim` denotes total width across heads rather than per-head width, keeping GAT's capacity comparable to GCN's and GraphSAGE's at the same `hiddenDim` rather than inflated by the head count.

`GcniiModel` bypasses `GnnModel.__init__` entirely, calling `nn.Module.__init__` directly, and wraps exactly `numLayers` `GCN2Conv` hops at constant `hiddenDim` width between an uncounted `Linear(1433, hiddenDim)` input projection and, under `LastLayerReadout`, an uncounted

`Linear(hiddenDim, 7)` output projection (Section 4.2.1). The projected input is never written into `layerEmbeddings`: index 0 remains the raw input `X` for GCNII exactly as for every other architecture.

5.3.3 `metrics`

`DirichletEnergy` is one row-scale, one gather by edge index, one squared-difference row-norm, and one sum: `[2708, hiddenDim]` in, one Python `float` out, for every layer in a run. The module's substance is three numerical guards. `MeanAverageDistance` clamps the row-norm denominator to `1e-12` before normalizing, since ReLU produces exact all-zero rows at depth and an unclamped division would produce `nan` rather than a defined cosine distance; it separately clamps both of MAD's own reduction denominators (Eq. 3's per-row count and Eq. 4's final count) to return `0.0` rather than `NaN` when a count is zero, which is what a fully collapsed representation (every row identical, every pairwise distance zero) produces. Without these guards the metrics would return `NaN` in exactly the regime the study measures. `FitContractionSlope` carries the third guard, the `1e-12` energy floor discussed in Section 4.5, needed because a genuinely collapsed configuration's per-dimension energy underflows to exact-zero in float32 and `log(0)` is undefined.

5.3.4 `train`

`TrainRun` is the module's public entry point; `EvaluateSplit`, `CaptureMetrics`, and `MacroF1` are its helpers, reused by the per-epoch validation pass and by the three capture points. The training loop itself (the `for epoch in range(1, config.maxEpochs + 1)` block) is 20-odd lines: forward, cross-entropy on `train_mask` rows, backward, optimizer step, then an eval-mode validation pass whose loss decides whether `bestState` is updated and whether the patience counter resets. Vectorization here is inherited rather than original to this module: the forward and backward passes vectorize because `models` and PyG's message-passing kernels do. `MacroF1` supplies its own: a `bincount` over the combined class-pair index `targets * numClasses + predictions`, reshaped into a confusion matrix from which the per-class counts are read as its diagonal and its row and column sums.

5.3.5 `mitigations`

`ResidualHook` and `PairNormHook` carry no learnable parameters and are plain Python objects rather than `nn.Module` subclasses; `JkReadout` does carry a `Linear` layer and is an `nn.Module`, since `GnnModel` stores `readout` as a direct attribute and PyTorch auto-registers it as a submodule, which is what lets that layer's parameters reach the optimizer alongside every convolution's. `JkReadout.Apply` asserts that every tensor in `layerEmbeddings[1:]` shares one width before pooling, so a misconfigured model would raise rather than silently broadcasting a shape mismatch into the pooling operation.

5.3.6 `experiments`

`BuildGrid` is six private `_BuildArm*` functions behind one dispatcher, each a small set of nested `for` loops over Python lists with no side effects: calling `BuildGrid` twice returns equal lists and touches no filesystem, which is what lets the grid's size be asserted directly in the test suite. `SetSeed(config.seed)` is called twice on the path from configuration to trained model (Figure 16 gives the exact call order). Weight initialization happens at model construction, which runs before `TrainRun`'s own seeding, so the earlier call in `experiments.runner` is what makes a run's initial weights follow `config.seed`.

`RunSweep` is idempotent, with atomic writes and exactly one writer: it skips any configuration whose result file already exists unless explicitly forced, records are written to a temporary path and renamed into place so a file is either complete or entirely absent, and `experiments` is the only module that ever writes to `results/`. At 534 runs over a single-machine, multi-day window, an interrupted sweep restarting from zero each time is the failure mode this design exists to avoid; a failing individual run is caught, written as a `<runId>.failed.json` marker alongside its error and traceback, and the sweep continues, so a genuine failure remains visible and distinguishable from a configuration that was simply never scheduled.

5.3.7 `viz`

Aggregation is strictly separated from plotting: `LoadRecords` reads and validates every JSON record in a directory (raising on a malformed file rather than skipping it silently, so a partially written or corrupted record surfaces immediately rather than quietly shrinking a seed count), `BuildTable` flattens each record to one tidy row, and `Aggregate` groups by a set of columns and returns mean, standard deviation, and count per group; only after this does any `Plot*` function touch `matplotlib`. Mean and standard deviation are computed at read time and never written back to `results/`, so the records directory remains the single source of truth with no derived file beside it that could go stale. The comparable band is read from each record's stored `bandIndices`, never recomputed by the plotting code, mirroring the derivation rule in Section 4.5 from the consuming side: a plotting function that assumed the `LastLayerReadout` band (`1..L-1`) would silently truncate every Jumping Knowledge curve by exactly one layer, which is the failure this design choice exists to prevent, and it is checked directly in the viz test suite (Section 5.4). Two plotting functions read a record directly rather than passing through `BuildTable` and `Aggregate`: `PlotEnergyVsLayer` and `PlotLossCurves`, which need one specific run's full per-layer or per-epoch array rather than a cross-seed aggregate.

5.4 Testing

5.4.1 Test Categories

The pytest suite under `src/tests/` replaces the "runs top to bottom without errors" guarantee a single assembled notebook would otherwise have to provide informally. Four categories of test recur across the seven test modules: numerical correctness against a hand-computable or externally verifiable reference (for instance, `metrics`' dense-reference energy test, which checks the vectorized edge-list computation against a dense `trace(H^T * Laplacian * H)` form on a small synthetic graph, and `train`'s macro-F1 test against `sklearn.metrics.f1_score`); grid enumeration and uniqueness (`experiments`' grid-arithmetic and no-duplicate-paths tests); shape and interface assertions (`models`' band-width-homogeneity and final-entry-identity tests, conditioned on the readout); and qualitative gates, which check a directional property on real, trained output rather than an exact value (the smoke test that a 2-layer GCN reaches roughly 81 percent test accuracy; the depth-32 collapse gate described next).

5.4.2 What Testing Caught

Two tests changed what this study reports, one by catching a defect and one by correcting an assumption about the data. The path-collision tests in `test_experiments.py` (`test_arm_e_and_f_collide_within_a_flat_directory` and `test_arm_f_still_collides_within_its_own_s` are what surfaced a filename collision between arms E/F and arm A, which the results filename

convention (Section 4.6) now avoids by giving each its own subdirectory. Without these tests, arm E's ten fidelity runs and several of arm F's hyperparameter-search runs would have silently overwritten arm A's result files, corrupting the depth-sweep aggregate with no error anywhere.

The collapse-floor tests in `test_metrics.py` (`test_collapse_floor_on_regular_graph` and `test_collapse_floor_on_cora_is_sqrt_degree_weighted`) caught no fault in the code; what they established is a property of the graph. Cora's actual null space is the `sqrt(degree)` direction rather than the literal constant vector, a consequence of the symmetric-normalized Laplacian on an irregular graph. An assumption that identical rows collapse to zero energy holds on a regular graph and would have been wrong on Cora, and Section 6.3 rests on the corrected reading.

5.4.3 The Epoch-0 Gate

One test is gated on the epoch-0 capture rather than the checkpoint capture, and the reason is itself a result. `test_depth_qualitative_gate_at_epoch_zero` asserts that a 32-layer unmitigated GCN shows monotonically decreasing per-dimension energy at initialization, not after training: at the checkpoint, under this study's settled hyperparameters, that monotonic property does not hold (Section 6.4 reports why), so gating the test there would make it flaky or outright false under a correct implementation. This connects directly to Section 6.4's central finding: separating "the propagation operator collapses representations" from "training collapses representations differently" is not only a results-section distinction, it is a distinction the test suite itself had to make to stay meaningful.

5.5 What the Code Does Not Do

No per-layer gradient norm and no per-layer weight norm is persisted anywhere in the results schema, for any run. `TrainRun` calls `.backward()` every training step but never captures or stores the resulting gradient magnitudes, and no run's trained weights are checkpointed to disk once training completes. Both Section 6 and Section 7 depend on this: it is the reason the mechanism behind GCN's depth-32 early-layer energy collapse (Section 6.4) remains an inferred explanation rather than a confirmed one, and it is the single instrumentation gap Section 7.2 identifies as most directly closing that gap if added.

6 Results and Discussion

Every number in this section is read from the `results/*.json` records or computed from them by `src/generate_report_figures.py` (Section 5). Results are reported as mean and standard deviation across 10 seeds (3 for the hyperparameter search) on the standard 140/500/1000 public Cora split, unless stated otherwise.

6.1 Two-Layer Baselines

Before anything about depth is claimed, our implementation is checked against results others have published. Each architecture is run at depth 2 under the shared configuration and compared against the figure its own paper reports (Section 3.19.5). Every energy or MAD figure and table below also names which of the three capture points (Section 3.12) it reads: epoch 0, the selected checkpoint, or the final epoch.

GCN reaches 81.49% (± 0.32), against 81.5% in Kipf and Welling and 81.4% (± 0.5) for the 64-width

variant in Veličković et al. GAT reaches 78.90% (± 1.57), against 83.0% (± 0.7), roughly four points short (Table 7), *consistent with* the shared configuration, from which GAT's own setup differs most (Section 6.9). GraphSAGE reaches 80.10% (± 0.42); Hamilton et al. evaluate on inductive benchmarks, so Cora is outside their reported results.

Table 7: Depth-2 test accuracy against the figure each architecture's own paper reports, ten seeds each. GraphSAGE has no published Cora result.

architecture	measured	std	published	difference
GCN	81.49%	0.32	81.5%	-0.01 pp
GraphSAGE	80.10%	0.42	none published	—
GAT	78.90%	1.57	83.0% (± 0.7)	-4.10 pp

Two checks support that comparison. The first is width: arm E at `hiddenDim=16` reaches 81.60% (± 0.31) against arm A's 81.49% (± 0.32) at 64, 0.11 points apart (Table 8), within one standard deviation of either, so the sweep's width does not move the depth-2 result.

Table 8: Depth-2 GCN test accuracy at the published width (`hiddenDim=16`, arm E) versus this study's sweep width (`hiddenDim=64`, arm A). Ten seeds each.

convType	hiddenDim	numLayers	testAccuracy_mean	testAccuracy_std	count
gcn	16	2	0.8160	0.0031	10
gcn	64	2	0.8149	0.0032	10

The second is the hyperparameter search, which selects Kipf and Welling's configuration exactly. It does not separate its eight candidates (Table 9): all eight span 1.14 accuracy points while per-configuration standard deviations, from three seeds, run 0.12 to 0.42. Ranked by mean validation accuracy over 3 seeds:

Table 9: Arm F, all eight hyperparameter combinations, GCN at depth 2, ranked by mean validation accuracy over 3 seeds.

learningRate	dropout	weightDecay	valAccuracy_mean	valAccuracy_std
0.01	0.5	0.0005	0.8047	0.0042
0.01	0.5	0.001	0.8000	0.0020
0.005	0.5	0.001	0.7993	0.0031
0.01	0.6	0.0005	0.7993	0.0031
0.01	0.6	0.001	0.7987	0.0023
0.005	0.6	0.0005	0.7980	0.0020
0.005	0.6	0.001	0.7967	0.0012
0.005	0.5	0.0005	0.7933	0.0042

The selection holds anyway: the winner (`learningRate=0.01, dropout=0.5, weightDecay=0.0005`)

is also the configuration closest to the published baseline, the search's fallback criterion, which would have chosen it on that basis alone.

6.2 Accuracy and Macro-F1 Across Depth

Test accuracy and macro-F1 against depth, for the unmitigated arm A sweep (three architectures) and arm C (GCNII, discussed on its own terms in Section 6.6):

Table 10: Test accuracy and macro-F1 versus depth, unmitigated GCN/GraphSAGE/GAT, ten seeds each.

convType	depth	testAccuracy_mean	testAccuracy_std	macroF1_mean	macroF1_std
gcn	2	0.8149	0.0032	0.8032	0.0032
gcn	4	0.7368	0.0656	0.7381	0.0495
gcn	8	0.3349	0.1162	0.2864	0.1013
gcn	16	0.2352	0.0351	0.2104	0.0309
gcn	32	0.2407	0.0160	0.2102	0.0161
sage	2	0.8010	0.0042	0.7917	0.0040
sage	4	0.7172	0.0463	0.7086	0.0479
sage	8	0.2722	0.1078	0.0902	0.0734
sage	16	0.2850	0.0717	0.0627	0.0135
sage	32	0.2265	0.0983	0.0514	0.0189
gat	2	0.7890	0.0157	0.7780	0.0153
gat	4	0.7450	0.0593	0.7391	0.0641
gat	8	0.3228	0.1005	0.1563	0.1540
gat	16	0.2845	0.0727	0.0626	0.0137
gat	32	0.1966	0.0846	0.0459	0.0160

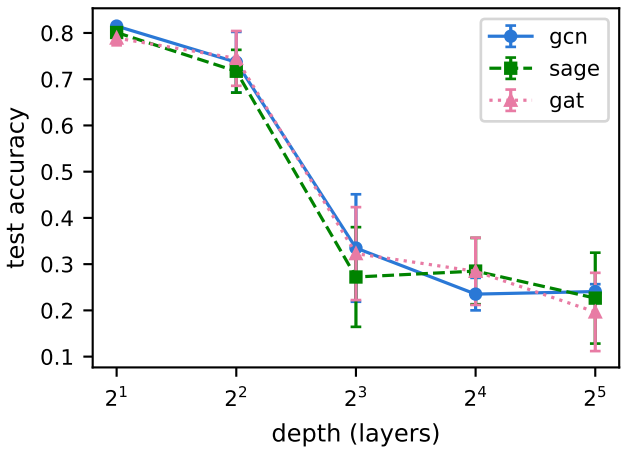


Figure 9: Test accuracy versus depth, unmitigated GCN, GraphSAGE, and GAT (arm A).

All three conventional architectures degrade with depth, and GCNII is the sole exception (Table 10, Figure 9; GCNII's own curve is deferred to Section 6.6). The degradation is not gradual across the

full grid: all three drop 4.4 to 8.4 points from their depth-2 accuracy at depth 4, then drop sharply between depths 4 and 8, and remain low through depth 32.

Macro-F1 collapses more sharply than accuracy for all three degrading architectures, most visibly for GraphSAGE and GAT. At depth 32, GAT's accuracy (19.66%) is roughly 5 points above the uniform-random floor for seven classes (14.3%, Section 3.19), while its macro-F1 (4.59%) is far below what either a uniform-random or majority-class predictor would produce on a balanced macro-averaged metric. This is *consistent with* predictions concentrating onto a small number of classes rather than spreading errors evenly across all seven.

The comparison against the majority-class floor (Section 3.19) holds for every seed, not only on average. The Cora test split's majority-class floor, computed directly from the test-mask label counts (`[130, 91, 144, 319, 149, 103, 64]`, summing to 1000), is 31.9%. At depth 32, unmitigated GCN's mean test accuracy (24.07%) sits below this floor, and its full 10-seed range (21.2%-26.2%) sits below it entirely: even GCN's single best seed is 5.7 points short of a predictor that always guesses the majority class.

6.3 Collapse at Initialization

This section measures the collapse before training begins, isolating the architecture from the optimizer: at epoch 0 the weights are random, so whatever contraction appears cannot be attributed to anything learned. GCN and GAT collapse cleanly and monotonically in both direction (MAD) and magnitude (energy ratio); GraphSAGE collapses in direction only (Table 11). What is measured is the initialized network as a whole, the propagation operator composed with each layer's randomly initialized weight matrices. The weights' magnitudes are part of that composition and contribute to the observed contraction: $\|W\|$ enters Cai and Wang's bound whether or not the weights have been trained (Section 3.18). GCN's energy ratio falls roughly 12 orders of magnitude by depth 32 and GAT's falls roughly 10, *consistent with* the repeated multiplicative contraction the bound describes, though its numerical tightness is not tested here. This measurement is the reference the trained state is read against (Section 6.4).

Table 11: Epoch-0 (pre-training) MAD and energy ratio versus depth, unmitigated GCN/GraphSAGE/GAT. Arithmetic means over ten seeds. At depth 2 the comparable band holds a single layer, so the energy ratio divides that layer's energy by itself and is not reported. GraphSAGE's MAD at depths 16 and 32 sits at the noise level around zero; the depth-32 standard deviation (0.041) exceeds the mean.

convType	metric	depth 2	depth 4	depth 8	depth 16	depth 32
gcn	MAD	0.600	0.240	0.085	0.023	0.0045
gcn	energy ratio (`E_last/E_1`)	—	3.14e-2	3.14e-4	2.60e-7	6.22e-13
gat	MAD	0.596	0.211	0.076	0.017	0.0041
gat	energy ratio (`E_last/E_1`)	—	7.26e-2	2.73e-3	1.28e-5	1.25e-10
sage	MAD	8.28e-2	1.83e-4	9.95e-7	-1.75e-7	-1.30e-2
sage	energy ratio (`E_last/E_1`)	—	18.3	19.8	20.3	19.3

6.3.1 GraphSAGE: MAD Against Energy

GraphSAGE's two metrics dissociate. Its MAD reaches near-zero by depth 4, faster than GCN or GAT, while its energy ratio *rises* from depth 2 to depth 4 and then plateaus around 19-20 at every depth tested.

A singular-value decomposition at depth 16, over three seeds, locates the cause. GraphSAGE's representation matrix has a top-singular-value energy fraction of 1.0000, an exact rank-1 matrix, not merely one dominated by a single direction. Its top singular vector matches the constant direction at cosine 1.0000, against 0.9512 to the $\sqrt{\text{degree}}$ direction. GCN and GAT are only mostly rank-1 (energy fraction 0.93-0.98) and do not cleanly separate the two candidate directions.

The augmented, symmetric-normalized Laplacian's null space on Cora is the $\sqrt{\text{degree}}$ direction, not the literal constant vector, because Cora's degrees run from 2 to 169 (Section 3.3, Section 3.20). GraphSAGE collapses onto the constant direction, which is therefore outside that null space, so its Dirichlet energy cannot vanish however completely its representations align. MAD measures whether representations point the same way, so it registers the collapse. Dirichlet energy measures how much they differ across edges, and that depends on *which* direction they collapse onto, so a collapse onto a direction outside the null space leaves energy high.

6.4 The Trained State

Training changes how MAD and the energy ratio behave as depth increases. At the selected checkpoint (Section 3.12.2), GCN's MAD no longer falls with depth and its energy ratio grows where it previously decayed. GraphSAGE and GAT still collapse on both. That difference between GCN and the other two is the subject of Section 6.5. Figure 10 shows that shift directly, tracking the same band index across all three captures and across depth.

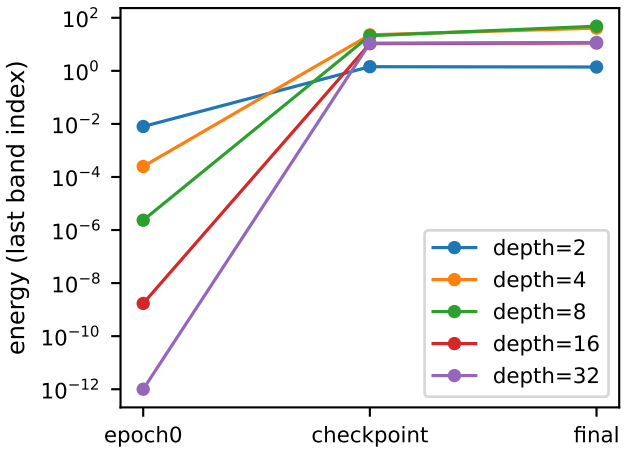


Figure 10: Per-dimension energy at the last comparable band index, across the three metric captures (epoch 0, checkpoint, final epoch) and across depth, unmitigated GCN. Geometric mean over ten seeds.

Checkpoint MAD at the last band index, mean over 10 seeds:

Table 12: Checkpoint (post-training) MAD versus depth, unmitigated GCN/GraphSAGE/GAT.

depth	GCN MAD	SAGE MAD	GAT MAD
2	0.240	0.262	0.286
4	0.264	0.291	0.302
8	0.290	0.058	0.115
16	0.176	0.000	0.000
32	0.183	0.000	0.000

6.4.1 Checkpoint MAD

GCN's checkpoint MAD is roughly flat and non-monotonic across the whole depth sweep, never approaching zero, while SAGE and GAT's collapses to essentially zero by depth 16 and stays there (Table 12). The fact that MAD registers their collapse rules out metric insensitivity as the reason GCN shows no such pattern.

6.4.2 Checkpoint Energy Ratio

GCN's checkpoint energy ratio grows with depth. Values are summarized by geometric mean: at depth 32 the ratio spans 26 orders of magnitude across the ten seeds (Table 13), so an arithmetic mean would be set almost entirely by whichever seed is largest.

Table 13: Checkpoint energy ratio versus depth, unmitigated GCN, geometric mean and range over 10 seeds. At depth 2 the comparable band holds a single layer, so the ratio is not reported.

depth	`E_last/E_1` geometric mean	range across 10 seeds
2	—	—
4	26.6	16.0-43.6
8	3.96e6	2.12e2-7.85e11
16	4.49e12	7.51e9-5.08e19
32	5.45e26	3.25e12-2.49e38

The ratio is enormous because the denominator has collapsed. `E_1` is approximately 1.4e-15 at depth 32, seed 0, with other seeds ranging down to 2.9e-38, already many orders of magnitude below a healthy energy value. The checkpoint state therefore shows early-layer collapse together with late-layer growth: the early band carries essentially zero energy while the late band carries enormous energy. Figure 11 shows the shape of that split across the depth-32 band.

No `inf` or `nan` appears in any of the ten seeds' recorded energies. The lower end of the range is where the float32 format binds. The smallest `E_1` (2.9e-38, seed 9) sits within a factor of 2.5 of float32's smallest normal value (1.18e-38), and for four of the ten seeds (2, 4, 7, 8) one band layer has already reached exactly 0.0. Part of what "essentially zero energy" describes in the early band is therefore the format's floor rather than a value the network resolved. This does not change the finding, since the layers on either side of those four carry measured values many orders of magnitude above zero, but the boundary is named rather than left implicit.

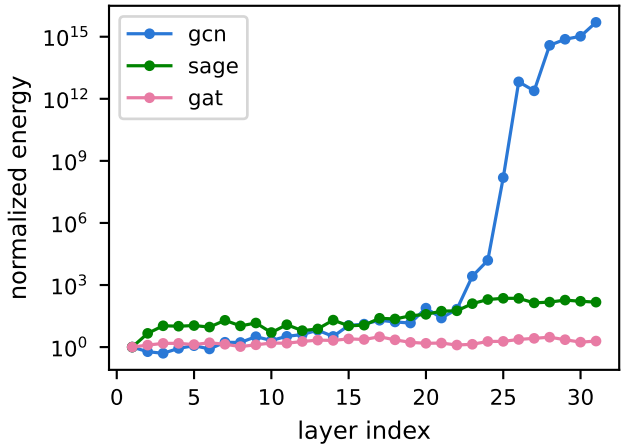


Figure 11: Normalized per-dimension energy versus layer index, checkpoint capture, depth 32, unmitigated GCN/GraphSAGE/GAT.

6.4.3 Cross-Seed Consistency

Early-layer collapse recurs across seeds. Per-seed count of band layers (of 31) whose energy falls below the `1e-12` floor at the checkpoint:

Table 14: Checkpoint below-floor band layers, unmitigated GCN, depth 32, per seed. For seeds whose energy oscillates across the floor, the first at-or-above layer precedes later below-floor layers, so the two counts do not differ by one.

seed	below-floor layers (of 31)	first layer at or above floor	`E_1`
0	22	23	1.4e-15
1	22	23	5.1e-20
2	26	27	7.5e-33
3	24	25	6.5e-24
4	25	25	2.9e-35
5	25	26	1.9e-31
6	0	1	3.3e-12
7	25	24	1.5e-26
8	25	22	4.2e-28
9	27	28	2.9e-38

Nine of ten seeds show the same shape (Table 14): roughly the first 22-28 layers carry essentially zero energy, with non-negligible energy only in the last 4-9. For three seeds (4, 7, 8) the transition oscillates across the floor before settling rather than crossing once. Seed 6 is the exception: no layer falls below the floor. It also has the highest test accuracy of the ten (26.2%) and the shortest run (315 epochs against 385-883), a correlation carried to Section 7.1.

6.4.4 The Inferred Mechanism

The mechanism behind this collapse (early-layer parameters starved of gradient signal while late-layer weights grow large and unconstrained) is *consistent with* the measured pattern above, but remains one-run inference, not a cross-seed measured fact. Terminal weight norms were inspected for exactly one run (`gcn_none_d32_s0`), post hoc, not as a persisted schema field and not as a gradient trace. No per-layer gradient norm exists anywhere in the recorded data for any run (Section 5.5), so this mechanism cannot be upgraded from *consistent with* to *measured* without new instrumentation (Section 7.2). Nothing further is claimed about "large and unconstrained": whether the late layers are fitting, or overfitting, the 140-node training set specifically is a separate, unmeasured claim. The schema has no training-accuracy field, and the measured test accuracy (24.07%, below the 31.9% majority floor) is in tension with overfitting.

6.5 Oversmoothing Against Optimization Failure

Accuracy alone cannot distinguish poor performance caused by a model that never trained from one that trained and then collapsed. At depth 32, both failure modes produce test accuracy near the majority-class floor; the results record's `trainingCurve` and `epoch0Metrics` fields exist specifically so the two are not conflated.

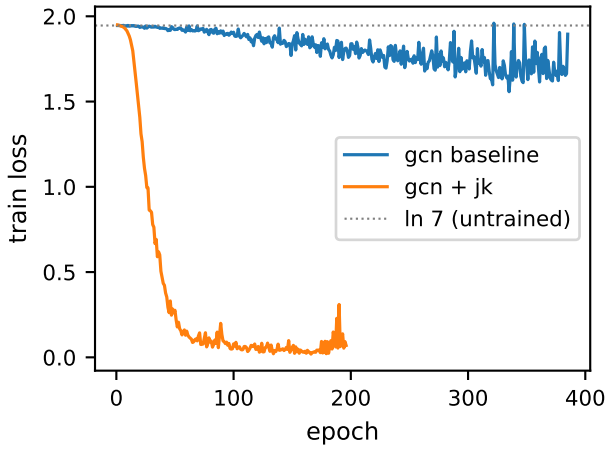


Figure 12: Training loss versus epoch, depth 32: unmitigated GCN baseline versus GCN with Jumping Knowledge.

6.5.1 Loss Trajectory and Epochs Run

The training loss separates the three: GCN partially trains, while GraphSAGE and GAT do not train at all. Across all 10 seeds, unmitigated GCN's training loss descends from the uniform-prediction loss `ln(7) = 1.9459` to a mean minimum of 1.57 (range 1.53-1.61), well above depth 2's well-fit final loss of approximately 0.14, but a clear, non-trivial descent (Figure 12). GraphSAGE and GAT's training loss stays flat at `ln(7)` (mean minimum 1.934 and 1.939 respectively, a movement of 0.01-0.02 nats, indistinguishable from optimization noise).

The independent stopping-behavior signal points the same way. GCN's runs last 315-883 epochs before `patience=100` exhausts, long enough that validation loss kept improving somewhere along

the way. GraphSAGE and GAT's runs exhaust patience almost immediately (102-168 and 101-121 epochs respectively), *consistent with* validation loss never improving past its epoch-0 value.

6.5.2 Converging Measurements

Three separately measured quantities converge for GraphSAGE and GAT: the flat training loss, the near-immediate exhaustion of patience, and the checkpoint's near-total MAD collapse (Section 6.4, essentially identical to the epoch-0 collapse in Section 6.3). Each is read directly from the record, and none is derived from the others, which is what makes their agreement convergence rather than circularity. That agreement is *consistent with* training essentially never moving GraphSAGE's and GAT's weights at depth 32.

6.5.3 Taxonomy of Depth-32 Behavior

Depth-32 failure separates on two axes. On training behavior there are two modes: GCN partially trains, reaching an accuracy plateau below the majority-class floor; GraphSAGE and GAT do not train at all, so their checkpoint state stays close to epoch 0. Within that non-training pair the collapse signatures still differ: GAT collapses on both metrics, GraphSAGE in direction only, its energy ratio rising rather than decaying (Section 6.3).

6.6 GCNII Across Depth

GCNII's test accuracy improves with depth from depth 4 onward, and its best result is at depth 32, the opposite direction from GCN, GraphSAGE, and GAT:

Table 15: GCNII test accuracy versus depth, ten seeds.

depth	testAccuracy_mean	testAccuracy_std
2	0.7845	0.0179
4	0.7581	0.0482
8	0.7766	0.0163
16	0.8016	0.0104
32	0.8326	0.0044

Depth 32 is GCNII's best result and has the tightest cross-seed spread of any depth tested (0.44 accuracy points, versus 1-5 points at every other depth in Table 15).

GCNII's checkpoint contraction slope (Section 3.17) shrinks toward zero as depth grows and stays near it, rather than growing as GCN's does:

Table 16: GCNII checkpoint contraction slope versus depth, ten seeds. Depth 2 omitted (single-point band, `nan`).

depth	checkpointContractionSlope_mean	checkpointContractionSlope_std
4	+0.383	0.063
8	+0.169	0.024
16	-0.037	0.010

depth	checkpointContractionSlope_mean	checkpointContractionSlope_std
32	+0.012	0.003

At depth 16 the mean is slightly negative (-0.037, std 0.010), and at depth 32 it returns to +0.012; across all depths tested it stays within roughly ±0.4 (Table 16). GCN's checkpoint slope under the same floored fitting procedure is larger at every comparable depth, running from +1.64 at depth 4 to +2.46 at depth 8 and +0.72 at depth 32, roughly an order of magnitude above GCNII's.

GCNII's identity-mapping and initial-residual terms are designed to prevent the vanishing-gradient dynamic Section 6.4 infers for GCN, and the unconstrained late-layer growth that produces it. The near-zero slope and the improving accuracy are *consistent with* that design working as intended. GCNII's weight norms and gradients were not inspected, so the mechanism stays a reading of the accuracy and slope pattern.

6.7 Mitigation Ablation

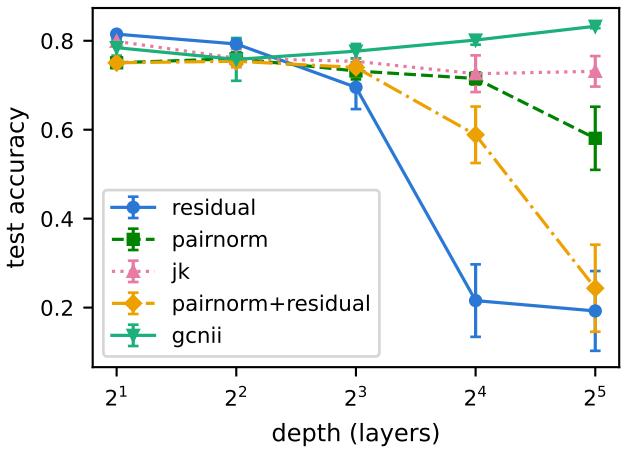


Figure 13: Test accuracy versus depth, GCN mitigation ablation (arm B) plus GCNII (arm C).

The full ablation at depth 32, all five arms including the unmitigated baseline:

Table 17: Mitigation ablation on GCN, depth 32, ten seeds each.

arm	testAccuracy_mean	testAccuracy_std	min	max
none (baseline)	24.07%	1.60	21.2%	26.2%
residual	19.25%	8.97	10.3%	32.1%
pairnorm	58.07%	7.10	47.6%	65.6%
jk	73.13%	3.43	68.0%	78.2%
pairnorm+residual	24.36%	9.79	13.5%	41.0%

6.7.1 The Full Ablation

Jumping Knowledge is the clear winner (Table 17), roughly 15 points ahead of the next best mitigation and with the tightest cross-seed spread of any mitigated arm, the basis on which it was selected to carry forward to GraphSAGE and GAT as arm D. PairNorm helps substantially (58.07% against the 24.07% baseline) but well short of JK, and Figure 13 shows the gap is a depth effect: JK's curve is nearly flat across the sweep while PairNorm's holds to depth 16 and then falls. Residual alone does not clearly beat the unmitigated baseline at depth 32: its mean (19.25%) sits below baseline's (24.07%), though its standard deviation (8.97) is large enough, and its maximum (32.1%) high enough, that this is not a confident "residual hurts" claim on ten seeds alone. Combining PairNorm with residual is worse than PairNorm by itself (24.36% versus 58.07%), a negative interaction between the two mechanisms at this depth, closer to the residual-alone and baseline numbers than to PairNorm's, rather than any combination of their individual benefits. Neither pattern is explained here. Both would need the per-run scrutiny given to the unmitigated baseline in Section 6.3 and Section 6.4, which no mitigated arm received; both are carried to Section 7.1.

6.7.2 Residual

A design note flagged the absent layer-1 residual as a candidate explanation should the residual arm underperform, and it did. Residual was designed, per Kipf and Welling's own Appendix B protocol, as a no-projection identity hook that cannot fire on the width-changing boundary layers (layer 1 and, under `LastLayerReadout`, the last layer), and the design note recorded at that time flagged explicitly that if the residual arm showed no benefit at depth, the absent layer-1 residual would be a candidate explanation. That candidate has not itself been checked (it would require a layer-1-projection diagnostic outside the current ablation), so this is the design's prediction confirmed as *worth investigating*, not confirmed as *correct*.

6.7.3 The Mitigation on GraphSAGE and GAT

Jumping Knowledge is the mitigation carried to GraphSAGE and GAT in arm D, selected on the depth-32 comparison above per the study's own selection rule (highest mean test accuracy at depth 32, not averaged across all five depths, since the ablation's purpose is restoring deep performance specifically). Arm D runs it on both architectures across the full depth grid, ten seeds each.

Table 18: Jumping Knowledge at depth 32, against the unmitigated baseline, ten seeds each. GCN's row is arm B, GraphSAGE's and GAT's arm D.

architecture	unmitigated	+JK	difference	unmitigated macro-F1	+JK macro-F1
GCN	24.07% (± 1.60)	73.13% (± 3.43)	+49.06 pp	0.2102	0.7249
GraphSAGE	22.65% (± 9.83)	40.47% (± 3.76)	+17.82 pp	0.0514	0.2611
GAT	19.66% (± 8.46)	72.09% (± 3.77)	+52.43 pp	0.0459	0.7252

The mitigation is architecture-dependent (Table 18). On GAT it recovers 52.43 points, to 72.09%

(± 3.77), which is within one standard deviation of GCN+JK's 73.13% (± 3.43); macro-F1 recovers to 0.7252 against GCN+JK's 0.7249. On GraphSAGE it recovers 17.82 points, to 40.47% (± 3.76). That is a real gain and all ten seeds sit above the 31.9% majority-class floor (range 33.3% to 46.4%), but it is roughly a third of what the same mitigation does for the other two, and macro-F1 reaches only 0.2611, so the recovered accuracy is not spread evenly across the seven classes.

Table 19: Test accuracy versus depth with Jumping Knowledge, all three architectures, ten seeds each.

depth	GCN+JK	GraphSAGE+JK	GAT+JK
2	79.87% (± 0.51)	78.52% (± 1.34)	80.28% (± 0.70)
4	76.08% (± 2.04)	71.86% (± 2.83)	76.14% (± 1.78)
8	75.37% (± 2.74)	67.27% (± 5.92)	75.63% (± 3.38)
16	72.59% (± 4.13)	55.25% (± 9.23)	71.87% (± 5.15)
32	73.13% (± 3.43)	40.47% (± 3.76)	72.09% (± 3.77)

The depth curve separates the three the same way (Table 19). GCN+JK and GAT+JK hold between 72% and 76% from depth 4 through depth 32; GraphSAGE+JK falls at every step, from 78.52% at depth 2 to 40.47% at depth 32. Jumping Knowledge delays GraphSAGE's degradation rather than preventing it.

GraphSAGE is also the architecture whose two metrics dissociate at epoch 0, collapsing in direction while its energy ratio rises (Section 6.3), and the one Jumping Knowledge does not rescue. Nothing here tests whether the two are related; the co-occurrence is recorded as an observation. No mechanism is proposed for why GraphSAGE resists the mitigation that works on the other two, and the question is carried to Section 7.1.

6.8 Embedding Projections

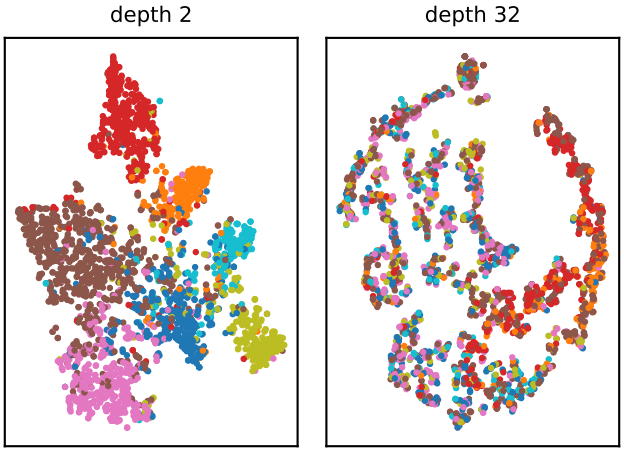


Figure 14: t-SNE projection of the checkpoint-capture layer embeddings, unmitigated GCN, seed 0, depth 2 versus depth 32, colored by class label. Fitted separately per panel; `random_state=0`, `perplexity=30`.

This projection illustrates collapse; the measurements are Dirichlet energy and MAD (Section 6.3, Section 6.4). The depth-2 panel shows seven visibly separated regions, each dominated by a single class label, consistent with that model's 81.49% test accuracy. In the depth-32 panel the class labels are intermixed almost everywhere: most local neighborhoods hold points of four or five different classes, and only one region, the arc along the right edge, stays visibly enriched in a single class. This is consistent with the 24.07% test accuracy at that depth.

What the panel does not show is a single collapsed point. The depth-32 embedding spreads across the panel in filaments rather than concentrating, and that agrees with the checkpoint MAD of 0.183 (Section 6.4), which says GCN's depth-32 representations remain directionally distinct rather than having aligned. The projection carries no information about absolute dispersion, since t-SNE rescales each panel independently, so the agreement is between the class intermixing the panel shows and the accuracy, not between the panel's spread and MAD. Read together, the two views separate loss of class information from collapse of the representations: at depth 32 the first has happened and the second, by MAD, has not.

6.9 Limitations

The results above hold within the following limits.

- Single dataset, single split. Every result in this section is specific to Cora's public 140/500/1000 split; nothing here establishes that the depth-failure taxonomy, the mitigation ranking, or the specific numbers generalize to another citation network, another homophily regime, or another split protocol.
- Findings that remain inferred rather than measured, named as such, with what would settle them: the parameter-level mechanism behind GCN's early-layer checkpoint collapse (Section 6.4; per-layer gradient norms would settle it, Section 7.2); the reason GraphSAGE's response to Jumping Knowledge differs from GCN's and GAT's (Section 6.7.3), since no mitigated arm received the per-run scrutiny given to the unmitigated baseline; and GCNII's own weight and gradient mechanism (Section 6.6), for the same reason.
- One shared hyperparameter configuration was used across four architectures with differing published setups. GAT and GCNII are the architectures furthest from the shared configuration, and GAT's depth-2 shortfall against its published figure (Section 6.1) is the visible cost.
- GCNII runs a hybrid configuration: its architectural hyperparameters (`alpha=0.1`, `theta=0.5`) are kept from the paper, while its training hyperparameters come from this study's shared configuration rather than the paper's own. This is not a full reproduction of the published GCNII result and is not presented as one.
- A "fully collapsed" representation under this study's metrics is not literally identical across nodes. Section 6.3 establishes that GCN and GAT's epoch-0 collapse is not onto the literal constant vector but onto (or near) the augmented Laplacian's actual null space, `sqrt(degree)`-weighted on Cora's irregular graph; a reader should not take "collapsed" in this report to mean "every node's representation is bitwise identical."

6.10 Applications

The real-world applications named in Section 1, recommendation systems, social-network analysis, and drug discovery, differ from Cora in ways that plausibly favor one of the three failure modes above over the others, though none of the three is tested directly on any of them.

Recommendation and social-network graphs built from user or member interactions typically have

degree distributions far more skewed than Cora's own (Section 3.20), with a small number of hub users or popular items accumulating a disproportionate share of edges, and are frequently too large for full-batch training. GraphSAGE's neighbor-sampling design, rather than the full-batch aggregation this study measures, is the form in which GraphSAGE is actually deployed at that scale. This study's finding that unmitigated GraphSAGE does not train at depth 32 on full-batch Cora is therefore not informative about GraphSAGE under sampling; the two settings differ in exactly the respect this study holds fixed, which neighbors enter each layer's aggregation.

Molecular graphs used for drug discovery are structurally the least similar of the three applications. Individual molecules are small, low-diameter, and close to regular: each atom bonds to a handful of neighbors rather than to the hub structure that makes Cora's own null space diverge from a constant vector (Section 3.3). The depth-32 failure modes measured here are therefore unlikely to be the binding constraint, and shallow-to-moderate depth is already closer to what molecular property prediction typically uses.

Which failure mode is most relevant to a given application is a question of how closely that application's graph resembles Cora's degree distribution and depth requirement. What this study's diagnostic apparatus offers these domains is the question its three capture points ask here, whether a deep model failed to train, or trained and then collapsed, asked on the application's own graph.

7 Recommendations for Future Work

Each recommendation below follows from an open question this study produced or from a quantity its records do not contain (Table 20).

7.1 Open Questions

- **GraphSAGE's single-hop collapse onto the constant direction.** Its MAD is already far below GCN's and GAT's at one hop, 0.083 against 0.600 and 0.596, and the separate root term is not the cause: MAD stays low, and lower, with `root_weight=False`. Two candidates remain, unisolated: unweighted mean aggregation interacting with Cora's skewed degrees, and `SAGEConv`'s `kaiming_uniform` initialization where GCN and GAT use Glorot (Paszke et al. 2019). A degree-normalized-mean variant and a Glorot-initialized `SAGEConv`, run separately, would separate them.
- **Why Jumping Knowledge rescues GAT but not GraphSAGE** (Section 6.7), and why residual underperforms baseline while PairNorm+residual underperforms PairNorm alone. No mitigated arm received the per-run scrutiny given to the unmitigated baseline.
- **Seed 6's exception** (Section 6.4): the only seed with no band layer below the energy floor, and also the highest test accuracy and shortest run of the ten. Whether the three facts share a cause was not examined.

7.2 What Was Not Measured

Table 20: Quantities absent from the records, and what each would settle.

Not measured	What it would settle	Why it is absent
Per-layer gradient norms	Whether early-layer parameters receive vanishing gradient signal, moving Section 6.4's mechanism from inferred to confirmed	Not persisted
Per-layer weight norms across the sweep	Whether the single-run weight-norm observation generalizes across seeds and architectures	Not persisted
A training-accuracy field	Whether GCN's deep late layers fit or overfit the 140-node training set	Not in the schema
Additional datasets differing in homophily and degree	Whether the `sqrt(degree)` null-space result (Section 6.3) is Cora-specific or general	Cora's degree range (2 to 169) is what makes the two candidate directions distinguishable
Per-architecture hyperparameter tuning	Each architecture's best achievable depth-32 performance	A different question; the shared configuration is held fixed so depth is not confounded with tuning (Section 4.6)

7.3 Scale and Noise

Cora tests neither scale nor noise (Section 2.7). Under sampling an L-layer receptive field only approximates the L-hop neighborhood, so this study's GraphSAGE result does not carry over; energy and MAD transfer unmodified, but whether they read each sampled subgraph or a fixed graph held aside needs a decision (Section 3.22). Under noise, accuracy alone cannot separate degraded label information from accelerated collapse, and the three capture points would show which.

Proposed improvement. PairNorm is the only tested mitigation that controls dispersion directly and adds no learned parameters, so its cost is independent of graph size. The contraction-slope results show the rate of representation change is not constant across depth, which motivates scaling PairNorm's strength with depth rather than fixing it. Not evaluated here.

Transfer to physics-informed graph learning. The numbers are not expected to carry to a mesh graph (Section 1); the method is: three capture points separating a trained from an untrained state before failure is attributed to oversmoothing, a band derived from the model's widths, and energy and MAD read as a pair.

References

Cai, Chen, and Yusu Wang. 2020. "A Note on over-Smoothing for Graph Neural Networks." *arXiv Preprint arXiv:2006.13318*. https://arxiv.org/abs/2006.13318.

Chen, Deli, Yankai Lin, Wei Li, Peng Li, Jie Zhou, and Xu Sun. 2020. "Measuring and Relieving the over-Smoothing Problem for Graph Neural Networks from the Topological View." *Proceedings of the AAAI Conference on Artificial Intelligence* 34 (4): 3438–45. https://doi.org/10.1609/aaai.v34i04.5747.

Chen, Ming, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. "Simple and Deep Graph Convolutional Networks." *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 119: 1725–35. https://proceedings.mlr.press/v119/chen20v.html.

Fey, Matthias, and Jan Eric Lenssen. 2019. "Fast Graph Representation Learning with PyTorch Geometric." *ICLR Workshop on Representation Learning on Graphs and Manifolds*. https://arxiv.org/abs/1903.02428.

Gilmer, Justin, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. "Neural Message Passing for Quantum Chemistry." *Proceedings of the 34th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 70: 1263–72. https://proceedings.mlr.press/v70/gilmer17a.html.

Hamilton, William L., Zhitao Ying, and Jure Leskovec. 2017. "Inductive Representation Learning on Large Graphs." *Advances in Neural Information Processing Systems* 30: 1024–34. https://papers.nips.cc/paper_files/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html.

Kipf, Thomas N., and Max Welling. 2017. "Semi-Supervised Classification with Graph Convolutional Networks." *5th International Conference on Learning Representations*. https://openreview.net/forum?id=SJU4ayYgl.

Li, Qimai, Zhichao Han, and Xiao-Ming Wu. 2018. "Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning." *Proceedings of the AAAI Conference on Artificial Intelligence* 32 (1): 3538–45. https://doi.org/10.1609/aaai.v32i1.11604.

Oono, Kenta, and Taiji Suzuki. 2020. "Graph Neural Networks Exponentially Lose Expressive Power for Node Classification." *8th International Conference on Learning Representations*. https://openreview.net/forum?id=S1ldO2EFPr.

Paszke, Adam, Sam Gross, Francisco Massa, et al. 2019. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in Neural Information Processing Systems 32*, 8024–35. https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.

Sen, Prithviraj, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. "Collective Classification in Network Data." *AI Magazine* 29 (3): 93–106. https://doi.org/10.1609/aimag.v29i3.2157.

Topping, Jake, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. 2022. "Understanding over-Squashing and Bottlenecks on Graphs via Curvature." *10th International Conference on Learning Representations*. https://openreview.net/forum?id=7UmjRGzp-A.

Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. "Graph Attention Networks." *6th International Conference on Learning Representations*. https://openreview.net/forum?id=rJXMpikCZ.

Xu, Keyulu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. "Representation Learning on Graphs with Jumping Knowledge Networks." *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of machine learning research, vol. 80: 5453–62. https://proceedings.mlr.press/v80/xu18c.html.

Yang, Zhilin, William W. Cohen, and Ruslan Salakhutdinov. 2016. "Revisiting Semi-Supervised Learning with Graph Embeddings." *Proceedings of the 33rd International Conference on Machine Learning*, Proceedings of machine learning research, vol. 48: 40–48. https://proceedings.mlr.press/v48/yanga16.html.

Zhao, Lingxiao. 2019. *PairNorm: Reference Implementation*. GitHub repository. https://github.com/LingxiaoShawn/PairNorm.

Zhao, Lingxiao, and Leman Akoglu. 2020. "PairNorm: Tackling Oversmoothing in GNNs." *8th International Conference on Learning Representations*. https://openreview.net/forum?id=rkecl1rtwB.

Appendix: System Diagrams

The two diagrams are a visual index into the code structure Section 5 describes in words.

Figure 15, the class diagram, follows on the next page, rotated to landscape for its width. It lays out `GnnModel` and its four architecture subclasses (`GcnModel`, `SageModel`, `GatModel`, `GcniiModel`), the two composition seams the base class exposes, `LayerHook` and `Readout`, and the concrete classes realizing each: `ResidualHook` and `PairNormHook` on the mitigation side, `LastLayerReadout` and `JkReadout` on the readout side. The dashed open-triangle arrows read as "realizes an interface"; the solid open-triangle arrows read as "inherits from"; the dashed "uses" arrows mark where `GnnModel`'s constructor wires a hook or a readout into the forward pass without the base class knowing its concrete type, the dependency-injection seam Section 5.3's `mitigations` entry describes in prose.

Figure 16, the sequence diagram, follows immediately after it. It traces one call to `RunOne`: seeding twice, once for weight initialization and once to isolate training-loop randomness such as dropout (Section 5.3's `experiments` entry), the epoch loop that alternates a forward pass with the validation-loss check driving early stopping, and the three `CaptureMetrics` calls, at epoch 0, at the final epoch before the best-checkpoint weights are restored, and at that restored checkpoint itself, that Section 3.12 defines and Section 4.4 explains the ordering of. The diagram makes visible a detail easy to miss reading the code sequentially: the final-epoch capture happens on the weights the training loop ended with, before `load_state_dict` restores the best-validation checkpoint, not after.

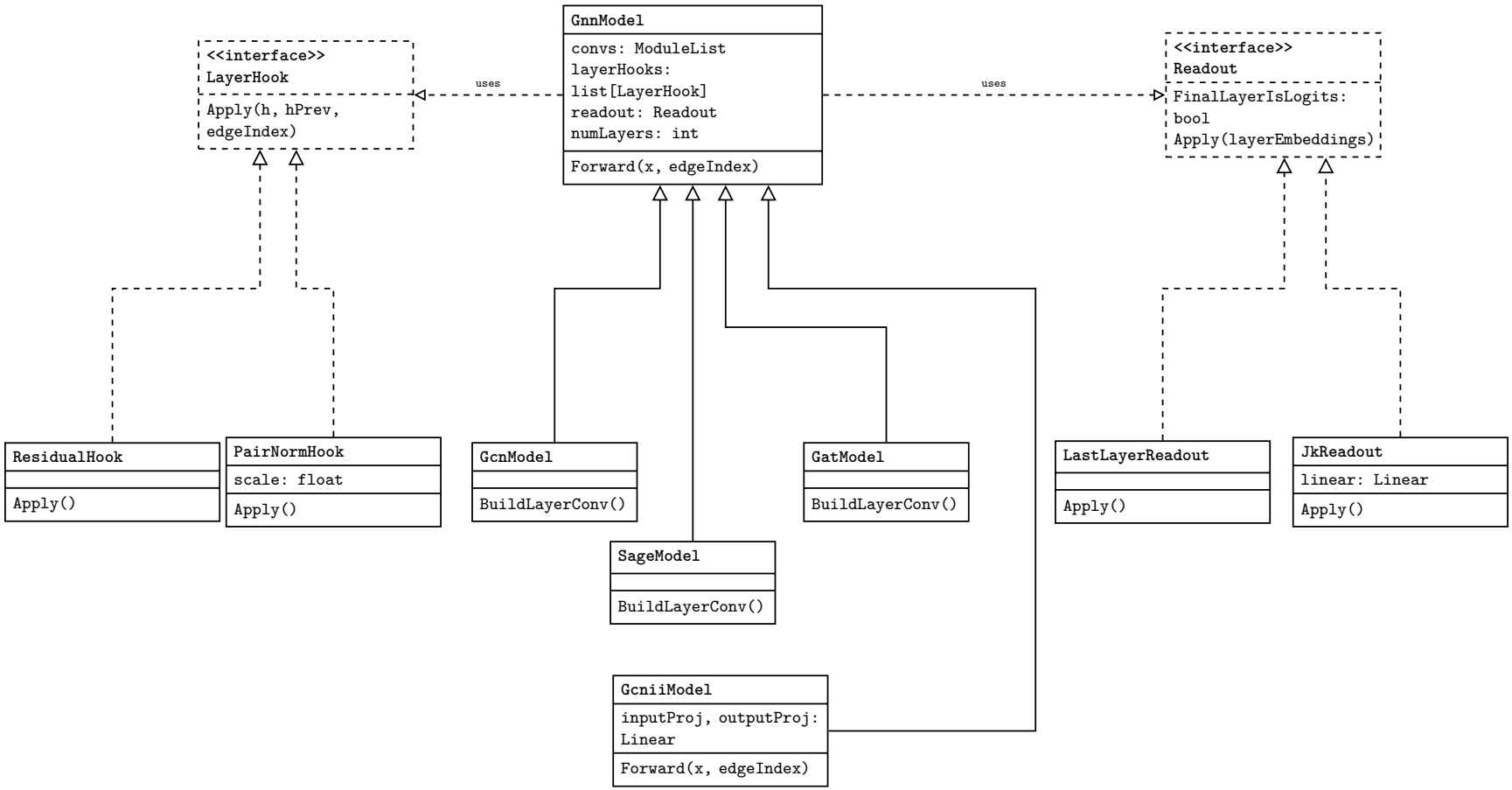


Figure 15: `GnnModel` and its four architecture subclasses (Section 5.3's `models` entry), the `LayerHook`/`Readout` composition seam `mitigations` implements against (Section 5.3's `mitigations` entry), and the two tested realizations of each protocol.

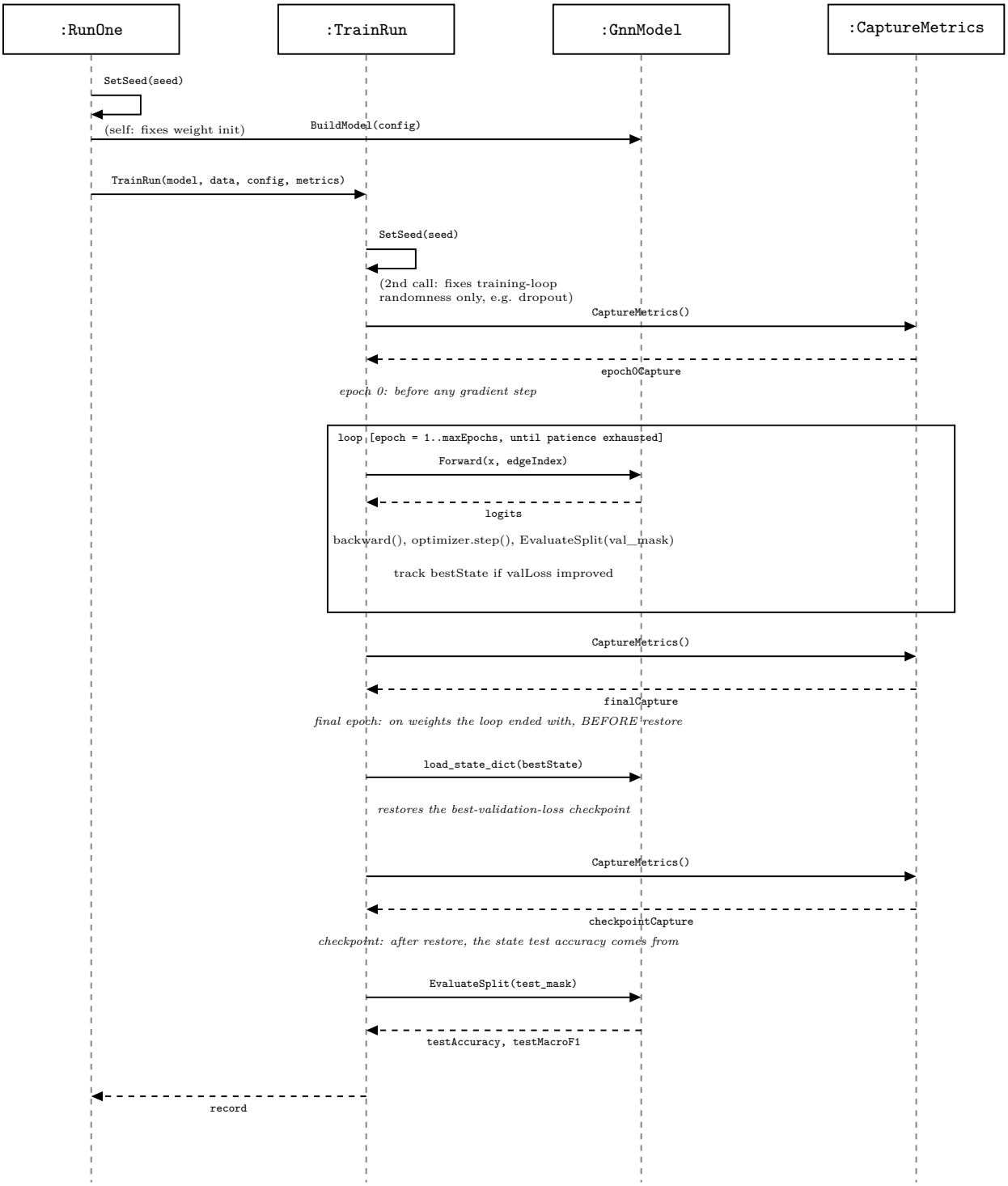


Figure 16: The call sequence for one run, from `RunOne` through the three metric captures (Section 3.12), showing the two `SetSeed` calls (Section 5.3's `experiments` entry) and that the final-epoch capture happens before, not after, the checkpoint restore (Section 4.4).